Documentación Técnica: Análisis Exploratorio de Datos (EDA)

Proyecto: Optimización de Inventario - Any Manufacturing Company

Responsable: Dalma Leguizamón

Fase: Etapa 4 - Diagnóstico y Análisis Estadístico

Objetivos del Análisis

El presente Análisis Exploratorio de Datos (EDA) tiene como finalidad transformar datos operativos previamente depurados en insights accionables para la toma de decisiones estratégicas dentro del sistema de inventario y abastecimiento de Any Manufacturing Company.

El estudio se desarrolla bajo un enfoque incremental, integrando información proveniente de ventas, compras, inventario y proveedores con el objetivo de comprender tanto el comportamiento de la demanda como el impacto del inventario sobre el capital de trabajo del negocio.

Los pilares principales del análisis se estructuran en tres dimensiones clave:

- **Identificación de Patrones de Demanda:** análisis temporal del comportamiento de ventas para entender la dinámica real del consumo y su concentración dentro del portafolio.

- **Salud del Inventario:** evaluación de la disponibilidad de stock mediante la detección de quiebres operativos (stockouts) y posibles excesos de inventario (overstock).

- **Priorización Estratégica:** implementación de una clasificación ABC basada en impacto económico para segmentar el catálogo de productos y orientar la gestión hacia los elementos de mayor relevancia financiera.

A lo largo del proceso, el análisis evoluciona desde la comprensión estructural del dataset hasta la valorización financiera del inventario, permitiendo traducir métricas operativas en conclusiones estratégicas sobre eficiencia, asignación de recursos y oportunidades de optimización.

Índice

Insights y Hallazgos Estratégicos ..... Hojas 3–5

SECCIÓN I – Infraestructura y extracción ..... Hojas 6–8

SECCIÓN II – Metodología de validación (Data Quality Check) ..... Hojas 9–14

SECCIÓN III – Ejecución del análisis por bloques estratégicos ..... Hojas 15–75

Bloque 1 – Comprensión General ..... Hojas 15–17

Bloque 2 – Demanda Implícita por Variación de Inventario ..... Hojas 18–20

Bloque 3 – Clasificación ABC ..... Hojas 21–24

Bloque 4 – Análisis de Rotación de Inventario ..... Hojas 25–36

Bloque 5 – Salud del Inventario ..... Hojas 37–49

Bloque 6 – Análisis de Lead Time y Abastecimiento ..... Hojas 50–69

Bloque 7 – Análisis Financiero del Inventario ..... Hojas 70–75

Insights y Hallazgos Estratégicos

Productos que concentran el capital

El análisis financiero muestra que el capital inmovilizado se encuentra principalmente concentrado en productos de categoría A, los cuales representan:

63,83% del capital total

Aproximadamente 35,3 millones USD del inventario valorizado.

Esto indica que el capital sigue la lógica estratégica del negocio: los productos que generan mayor impacto económico también concentran la mayor inversión.

Sin embargo, la categoría C todavía representa:

15,51% del capital total

Más de 8,5 millones USD, lo cual refleja oportunidades de optimización financiera en productos de menor relevancia.

sobrestock

No se identificó un problema estructural de sobrestock a nivel global.

El análisis del Bloque 5 ya indicaba que:

Solo un porcentaje mínimo del portafolio cae en condiciones de sobrestock extremo.

El inventario se encuentra relativamente alineado con la demanda.

Sin embargo, desde la perspectiva financiera (Bloque 7), se observa que:

31,12% del capital total está asociado a productos de baja rotación.

Esto no implica exceso estructural, pero sí una acumulación de capital en productos que tardan más tiempo en convertirse nuevamente en liquidez.

Productos que deberían reducir compra

Los productos que deberían revisarse prioritariamente son aquellos que cumplen simultáneamente:

Baja rotación (≤ 3).

Presencia dentro de categoría C.

Capital inmovilizado significativo.

El análisis muestra que:

Más de 6,89 millones USD del capital lento se encuentran en productos C.

Esto sugiere que las oportunidades de reducción de compra se concentran principalmente en este segmento, donde pequeñas optimizaciones pueden liberar capital sin afectar significativamente el revenue.

Productos que requieren mayor reposición

Los resultados del Bloque 5 indican que:

El riesgo crítico (alto consumo sin stock) es bajo en volumen total.

Sin embargo, cuando aparece, afecta principalmente a las categorías A y B.

Esto implica que los productos estratégicos deben continuar bajo:

Monitoreo activo.

Políticas de reposición frecuentes.

Control de cobertura ajustado.

No se detecta una falta generalizada de reposición, sino casos puntuales que requieren seguimiento operativo.

Sucursales que están están desbalanceadas

El análisis de tiendas muestra que:

41 de las 80 sucursales concentran el 80% del capital.

La mayoría de las tiendas se ubican en cuadrantes equilibrados (DemandaAlta + StockAlto o DemandaBaja + StockBajo).

Los desbalances observados son puntuales:

Tiendas con alta demanda y bajo stock → potencial riesgo operativo.

Tiendas con baja demanda y alto stock → oportunidad de optimización financiera.

Esto sugiere que las diferencias entre sucursales responden más a dinámicas locales que a problemas logísticos estructurales.

Recomendaciones Estratégicas Derivadas del Análisis

- Optimización de categoría C

Reducir progresivamente la compra en productos C con baja rotación, priorizando aquellos con mayor capital inmovilizado relativo.

Esto permitiría:

Liberar working capital.

Reducir permanencia en inventario.

Mejorar eficiencia financiera sin afectar significativamente el revenue.

- Ajuste de políticas de reabastecimiento por sucursal

Implementar ajustes finos en tiendas identificadas dentro de los cuadrantes:

DemandaAlta + StockBajo.

DemandaBaja + StockAlto.

El objetivo es mejorar la asignación regional del inventario sin aumentar el volumen total de compras.

- Control más estricto sobre productos categoría A

Dado que concentran más del 60% del capital:

Mantener monitoreo frecuente.

Ajustar cobertura para evitar quiebres sin sobredimensionar stock.

Los productos A representan el principal vector financiero del inventario.

- Revisión de frecuencia de compra para productos lentos

El análisis muestra que una proporción relevante del capital se encuentra en productos con baja rotación.

Se recomienda:

Evaluar reducción de frecuencia de compra.

Consolidar pedidos.

Evitar reposiciones fragmentadas en productos de baja dinámica.

SECCIÓN I: Infraestructura y extracción

1. Arquitectura de Conexión y Extracción

Para garantizar la trazabilidad y seguridad de la información, se implementó un puente de comunicación directo entre el entorno de análisis (Python) y el motor de base de datos.

Se ha diseñado un proceso de extracción robusto que garantiza la integridad de los datos y la eficiencia del entorno de desarrollo. El flujo se divide en cuatro fases críticas:

- **Fase A: Configuración de Entorno y Dependencias:**
  - Carga de librerías core: `pyodbc` para la interfaz de base de datos y `pandas` para el modelado de estructuras de datos (DataFrames).
  - Verificación del **ODBC Driver 17**, asegurando la compatibilidad con el motor SQL Server local.

```
# Importa librerías para conexión ODBC y análisis en DataFrames.
import pyodbc
import pandas as pd
```
✓ 5.3s

- **Fase B: Abstracción de Parámetros de Infraestructura:**
  - Implementación de un **Diccionario de Configuración (`config`)** para centralizar el nombre del servidor (`LAPTOP-MCIEH71J`) y la base de datos objetivo. Esto permite la portabilidad del código entre diferentes entornos (Local/Cloud) sin alterar la lógica principal.

```
# Definición de parámetros de infraestructura local.
# Se utiliza un diccionario para centralizar la configuración, facilitando la
# portabilidad entre entornos de desarrollo y producción.
config = {
    'server': 'localhost',
    'database': 'ManufacturingCompany'
}
```
✓ 0.0s

- **Fase C: Construcción de la Cadena de Conexión Dinámica:**
  - Uso de **f-strings** para ensamblar una `conn_str` técnica.
  - **Seguridad:** Uso de `Trusted_Connection=yes` (SSPI), delegando la autenticación al sistema operativo para evitar el "hardcodeo" de credenciales sensibles.
  - **Control de Concurrencia:** Inserción de un parámetro de `Timeout=30` para mitigar bloqueos en el Kernel de Python en caso de fallos de red o servicios apagados.

```python
# Construcción de la cadena de conexión (Connection String).
# 1. Trusted_Connection=yes: Implementa Autenticación de Windows (SSPI),
#    eliminando la necesidad de hardcodear credenciales sensibles (UID/PWD).
# 2. Timeout=30: Define un umbral de espera crítico para evitar el bloqueo
#    del Kernel de Python si el servidor no responde.
conn_str = (
    f"DRIVER={{ODBC Driver 17 for SQL Server}};"
    f"SERVER={config['server']};"
    f"DATABASE={config['database']};"
    f"Trusted_Connection=yes;"
    f"Timeout=30;"
)
```
✓ 0.0s

- **Fase D: Gestión de Conexiones y Control de Errores (Modularidad):**
  - Creación de la función `conectar_sql()`, la cual encapsula la lógica de apertura de sesión.
  - **Diagnóstico Preventivo:** El flujo incluye una red de seguridad mediante bloques `try-except` especializados que distinguen entre errores de driver (`InterfaceError`) y errores de acceso/red (`OperationalError`), facilitando el mantenimiento proactivo del pipeline.

```python
# Abre una conexión a SQL Server usando pyodbc.
# Devuelve el objeto connection si tiene éxito; si falla, devuelve None.

def conectar_sql():

    try:
        conn = pyodbc.connect(conn_str)
        print("✅ Conexión establecida con éxito.")
        return conn

    except pyodbc.InterfaceError:
        print("❌ Driver ODBC no encontrado (revisar versión 17/18).")

    except pyodbc.OperationalError:
        print("❌ No se pudo conectar (servidor apagado/credenciales/puerto).")

    except Exception as e:
        print(f"❌ Error inesperado: {e}")

    return None
```
✓ 0.0s

2. Diccionario de Datos de Usuario (Tablas Identificadas)

Se validó la existencia de 6 entidades críticas en el esquema `dbo` de la base de datos `ManufacturingCompany`:

1. **Ventas (`SalesDetail`):** Datos históricos de transacciones para el análisis de patrones de demanda.
2. **Inventario (`Inventory`):** Foto actual del stock on Hand para detección de quiebres y excesos.

3. **Compras (`Purchases`):** Órdenes de reabastecimiento para el cálculo de Lead Times de abastecimiento.
4. **Marcas (`Brands`):** Maestro de productos y categorías necesario para la segmentación y clasificación ABC.
5. **Sucursales (`Stores`):** Información sobre los puntos de venta para el análisis de distribución geográfica.
6. **Proveedores (`Proveedor`):** Identificación de fuentes de suministro para el análisis de confiabilidad de la cadena.

```python
# Verificamos y filtramos específicamente por el esquema 'dbo' (donde están tus tablas)
cursor = conexion.cursor()
for row in cursor.tables(schema='dbo'):
    if row.table_type == 'TABLE':
        print(f"📦 Tabla de usuario: {row.table_name}")
```

✓ 0.2s

SECCIÓN II: Metodología de Validación (Data Quality Check)

Antes de iniciar el Análisis Exploratorio de Datos (EDA), se realizó una revisión estructural del modelo con el objetivo de asegurar consistencia entre:

- El esquema definido en SQL Server (fuente estructural oficial).
- La interpretación de los datos al ser leídos en el entorno analítico (Python – Pandas).

1 Revisión del esquema en SQL Server

Se inspeccionaron las seis tablas del modelo relacional directamente desde la base de datos para validar:

- Nombre de las columnas.
- Tipo de dato declarado.
- Restricciones de anulabilidad.
- Integridad estructural general.

```python
def esquema_sql(conexion, tablas, schema="dbo"):
    filas = []
    for t in tablas:
        q = f"""
        SELECT
            '{t}' AS Tabla,
            c.COLUMN_NAME AS Columna,
            c.DATA_TYPE AS Tipo_SQL,
            c.IS_NULLABLE AS Permite_Null,
            c.CHARACTER_MAXIMUM_LENGTH AS MaxLen,
            c.NUMERIC_PRECISION AS Precision,
            c.NUMERIC_SCALE AS Scale,
            c.ORDINAL_POSITION AS Posicion
        FROM INFORMATION_SCHEMA.COLUMNS c
        WHERE c.TABLE_SCHEMA = '{schema}'
          AND c.TABLE_NAME = '{t}'
        ORDER BY c.ORDINAL_POSITION;
        """
        df = pd.read_sql(q, conexion)
        filas.append(df)

    out = pd.concat(filas, ignore_index=True)

    # Tipo SQL "completo" (ej: decimal(10,2), nvarchar(255))
    def tipo_sql_completo(row):
        t = row["Tipo_SQL"]
        if t in ("varchar", "nvarchar", "char", "nchar") and pd.notna(row["MaxLen"]):
            return f"{t}({int(row['MaxLen'])})"
        if t in ("decimal", "numeric") and pd.notna(row["Precision"]) and pd.notna(row["Scale"]):
            return f"{t}({int(row['Precision'])},{int(row['Scale'])})"
        return t

    out["Tipo_SQL_Completo"] = out.apply(tipo_sql_completo, axis=1)
    out = out[["Tabla", "Posicion", "Columna", "Tipo_SQL_Completo", "Permite_Null"]]
    return out

sql_schema = esquema_sql(conexion, TABLAS)
print("\n" + "="*80)
print("ESQUEMA EN SQL SERVER (dbo) — columnas y tipos reales")
print("="*80)
```
✓ 0.8s

El modelo relacional presenta tipado correcto y consistente.
Los tipos definidos en SQL (INT, DECIMAL, NVARCHAR, DATE, etc.) están correctamente estructurados y alineados con el diseño del modelo dimensional.

Se establece SQL Server como **fuente de verdad estructural (single source of truth)** para el tipo de datos.

2 Revisión de interpretación en Pandas

Posteriormente, se extrajeron muestras controladas hacia Python para evaluar cómo Pandas interpreta automáticamente los tipos de datos.

```python
def esquema_pandas(conexion, tablas, schema="dbo", sample_rows=200):
    filas = []
    for t in tablas:
        q = f"SELECT TOP ({sample_rows}) * FROM {schema}.{t};"
        df = pd.read_sql(q, conexion)  # trae pocas filas (no revienta memoria)

        tmp = pd.DataFrame({
            "Tabla": t,
            "Columna": df.columns,
            "Tipo_Pandas": [str(x) for x in df.dtypes]
        })
        filas.append(tmp)

    out = pd.concat(filas, ignore_index=True)
    return out

pandas_schema = esquema_pandas(conexion, TABLAS, sample_rows=200)
print("\n" + "="*80)
print("ESQUEMA EN PANDAS — cómo quedan los dtypes al traer una muestra")
print("="*80)
display(pandas_schema)
```

Se observaron las siguientes conversiones automáticas:

Tipo en SQL	Interpretación en Pandas
INT	int64
DECIMAL	float64
NVARCHAR	object
DATE	object

Hallazgos clave:

- Los tipos numéricos enteros se interpretan correctamente como `int64`.
- Los tipos decimales se convierten en `float64`, lo cual es aceptable pero puede implicar precisión flotante.
- Los tipos `NVARCHAR` se interpretan como `object`, lo cual es esperado.
- Los tipos `DATE` se interpretan como `object`, y no como `datetime64`, lo que requiere conversión explícita para análisis temporal correcto.

3 Riesgo Detectado

Se identificó que la conversión automática de Pandas puede generar:

- Interpretación incorrecta de fechas.
- Inconsistencias en cálculos temporales.
- Potenciales errores en análisis posteriores si no se controla el tipado.

Dado que el EDA implica cálculos de rotación, lead time, estacionalidad y análisis temporal, el control de tipos es crítico.

4 Decisión Técnica Adoptada

Para garantizar consistencia estructural durante todo el análisis:

Se definirá un diccionario de tipado (Data Type Mapping Dictionary) basado en el esquema oficial de SQL.

Este diccionario permitirá:

- Forzar el tipo correcto al momento de extraer datos desde SQL.
- Convertir explícitamente columnas de fecha a `datetime64`.
- Mantener consistencia analítica en todos los bloques del EDA.
- Evitar depender de la inferencia automática de Pandas.

De esta manera:

- SQL define la estructura.
- Python respeta la estructura.
- El análisis se ejecuta bajo tipado controlado.

```python
def tipos(tabla, conexion):
    """
    Trae una tabla desde SQL Server y ajusta los tipos de datos
    en pandas según el esquema real definido en SQL.
    """

    # Traemos los datos completos de la tabla
    query_datos = f"SELECT * FROM {tabla}"
    df = pd.read_sql(query_datos, conexion)

    # Consultamos el esquema real de la tabla en SQL
    query_schema = f"""
    SELECT COLUMN_NAME, DATA_TYPE
    FROM INFORMATION_SCHEMA.COLUMNS
    WHERE TABLE_NAME = '{tabla}'
    """
    esquema = pd.read_sql(query_schema, conexion)

    # Recorremos cada columna y ajustamos el tipo en pandas
    for _, fila in esquema.iterrows():
        columna = fila["COLUMN_NAME"]
        tipo_sql = fila["DATA_TYPE"]

        if columna in df.columns:

            # Enteros
            if tipo_sql in ["int", "bigint", "smallint", "tinyint"]:
                df[columna] = df[columna].astype("Int64")

            # Decimales
            elif tipo_sql in ["decimal", "numeric", "float", "real"]:
                df[columna] = df[columna].astype("float64")

            # Fechas
            elif tipo_sql in ["date", "datetime", "datetime2"]:
                df[columna] = pd.to_datetime(df[columna], errors="coerce")

            # Texto
            elif tipo_sql in ["varchar", "nvarchar", "char", "nchar", "text"]:
                df[columna] = df[columna].astype("string")

    # Devolvemos el DataFrame ya corregido
    return df
```

5. Control de Integridad y Hallazgo Crítico

Una vez implementado el **Diccionario de Tipado** y ejecutada la función para forzar la estructura de SQL en Python, se realizó un control de calidad sobre los DataFrames resultantes.

- **Detección:** Al aplicar el tipado estricto, el entorno de Python reveló la presencia de valores nulos (`<NA>`, `NaT`) en columnas críticas que debían contener datos íntegros.
- **Verificación en Origen:** Ante este hallazgo, se procedió a realizar una consulta directa (Query) en el motor de **SQL Server** para auditar la tabla directamente en la base de datos.
- **Confirmación del Error:** Se confirmó que la base de datos de SQL Server presentaba inconsistencias de carga (registros vacíos y columnas sin datos), determinando que el error de integridad reside en el almacenamiento de la base de datos y no en el código de procesamiento.

```python
# Llamamos a tu función
df_test = tipos("Inventory", conexion)

# 1. Ver los primeros datos
print("Visualización de datos:")
display(df_test.head())

# 2. Verificar que los tipos coincidan con tu lógica
print("\nVerificación de tipos forzados:")
print(df_test.dtypes)
```

```
Visualización de datos:
```

	InventoryID	StoreID	BrandID	OnHandBegin	OnHandEnd	StartDate	EndDate
0	1_HARDERSFIELD_1000	1	1000	0	<NA>	2016-01-01	NaT
1	1_HARDERSFIELD_1001	1	1001	0	0	2016-01-01	2016-12-31
2	1_HARDERSFIELD_10021	1	10021	25	23	2016-01-01	2016-12-31
3	1_HARDERSFIELD_1003	1	1003	<NA>	0	NaT	2016-12-31
4	1_HARDERSFIELD_1004	1	1004	17	<NA>	2016-01-01	NaT

```
Verificación de tipos forzados:
InventoryID    string[python]
StoreID                 Int64
BrandID                 Int64
OnHandBegin             Int64
OnHandEnd               Int64
StartDate      datetime64[ns]
EndDate        datetime64[ns]
dtype: object
```

	Tabla	Columna	Total_Filas	Cantidad_Nulos	Porcentaje_Nulos
1	Brands	Volume	12255	4	0.03
2	Brands	Size	12255	1	0.01
3	Brands	PurchasePrice	12255	0	0.00
4	Brands	RetailPrice	12255	0	0.00
5	Brands	BrandDescription	12255	0	0.00
6	Brands	BrandID	12255	0	0.00
7	Brands	Classification	12255	0	0.00
8	Brands	VendorNumber	12255	0	0.00
9	Inventory	StartDate	252450	45924	18.19
10	Inventory	OnHandBegin	252450	45924	18.19
11	Inventory	EndDate	252450	31550	12.50
12	Inventory	OnHandEnd	252450	31550	12.50
13	Inventory	StoreID	252450	0	0.00
14	Inventory	InventoryID	252450	0	0.00
15	Inventory	BrandID	252450	0	0.00
16	Provee...	VendorName	126	0	0.00
17	Provee...	VendorNumber	126	0	0.00
18	Purcha...	Freight	2372474	2372474	100.00
19	Purcha...	Approval	2372474	2372474	100.00
20	Purcha...	VendorNumber	2372474	0	0.00

6. Decisión Ejecutiva de Finalización

Para garantizar que el **Análisis Exploratorio de Datos (EDA)** se base en información 100% veraz y para no comprometer la **fecha estimada de entrega** del proyecto académico, se ha tomado la siguiente resolución:

> **Uso de Fuentes Primarias:** El análisis se ejecutará utilizando directamente los archivos **CSV originales**.

Esta decisión permite optimizar los tiempos de desarrollo, evitando el proceso de saneamiento manual de la base de datos (Data Cleaning en SQL), asegurando que los resultados estadísticos, cálculos de rotación y KPIs de inventario sean precisos y reflejen la realidad operativa del caso de estudio.

SECCIÓN III: Ejecución del Análisis por Bloques Estratégicos

Para transformar la infraestructura en decisiones de negocio, el análisis se despliega en 8 fases incrementales:

Bloque 1 – Comprensión General

En esta primera etapa se realizó un análisis general del dataset con el objetivo de entender la dimensión real del sistema antes de avanzar hacia evaluaciones más técnicas.

Primero se calculó la cantidad total de productos (SKUs) presentes en la base, identificándose **12.261 productos distintos**. Este número permite dimensionar la complejidad del portafolio y anticipar el nivel de dispersión que puede tener la demanda.

```
# Número total de productos
Precios_compras_2017["Brand"] = pd.to_numeric(Precios_compras_2017["Brand"], errors="coerce")
print("Número total de productos:", Precios_compras_2017["Brand"].nunique(dropna=True))
✓ 0.0s

Número total de productos: 12261
```

Luego se analizó la estructura operativa del negocio, determinando que el sistema está compuesto por **79 sucursales**, lo que implica una operación descentralizada y, por lo tanto, mayor complejidad en la gestión del inventario.

```
# # Número total de productos
Precios_compras_2017["Brand"] = pd.to_numeric(Precios_compras_2017["Brand"], errors="coerce")
print("Número total de productos:", Precios_compras_2017["Brand"].nunique(dropna=True))
Venta_final["Store"] = pd.to_numeric(Venta_final["Store"], errors="coerce")
print("Número de sucursales:", Venta_final["Store"].nunique(dropna=True))
✓ 0.0s

Número total de productos: 12261
Número de sucursales: 79
```

También se evaluó el período total cubierto por el dataset. Se identificó que la información abarca desde el **1 de enero de 2016 hasta el 10 de enero de 2017**, cubriendo aproximadamente **375 días de operación**, lo que permite realizar análisis anuales con consistencia temporal.

```python
# Calcular extremos globales (ignorando NaT)
fechas_min = [
    Inicio_inventario["startDate"].min(),
    Final_inventario["endDate"].min(),
    Facturas_compras["InvoiceDate"].min(),
    Compra_final["ReceivingDate"].min(),
    Venta_final["SalesDate"].min()
]

fechas_max = [
    Inicio_inventario["startDate"].max(),
    Final_inventario["endDate"].max(),
    Facturas_compras["InvoiceDate"].max(),
    Compra_final["ReceivingDate"].max(),
    Venta_final["SalesDate"].max()
]

fecha_inicio = pd.Series(fechas_min).dropna().min()
fecha_fin    = pd.Series(fechas_max).dropna().max()

duracion = fecha_fin - fecha_inicio

print("PERÍODO TOTAL CUBIERTO POR EL DATASET")
print(f"◆ Inicio: {fecha_inicio.date()}")
print(f"◆ Fin:    {fecha_fin.date()}")
print(f"◆ Duración total: {duracion.days} días")
```

✓ 0.0s

```
PERÍODO TOTAL CUBIERTO POR EL DATASET
◆ Inicio: 2016-01-01
◆ Fin:    2017-01-10
◆ Duración total: 375 días
```

En términos de volumen comercial, se calcularon las ventas totales del período, obteniendo:

- **2.451.169 unidades vendidas**
- **USD 33.139.375,29 en facturación**

```python
# Cálculo de volumen total
total_unidades = Venta_final["SalesQuantity"].sum()
total_facturacion = Venta_final["SalesDollars"].sum()

print("Volumen total vendido (unidades): {:,.0f}".format(total_unidades))
print("Volumen total vendido (USD): ${:,.2f}".format(total_facturacion))
```

✓ 0.0s

```
Volumen total vendido (unidades): 2,451,169
Volumen total vendido (USD): $33,139,375.29
```

Posteriormente, se cuantificó el inventario total al cierre del período, identificando:

- **4.885.776 unidades en stock**
- **USD 55.429.569,00 en capital inmovilizado**

```python
# Cálculo de inventario total
inventario_total = Final_inventario["onHand"].sum()

print("Volumen total de inventario (unidades): {:,.0f}".format(inventario_total))
```

✓ 0.0s

```
Volumen total de inventario (unidades): 4,885,776
```

```python
# Precio representativo por producto (mediana histórica)
precio_por_brand = (
    Precios_compras_2017
    .groupby("Brand")["PurchasePrice"]
    .median()
)

# Mapear costo al inventario actual
Final_inventario["CostoUnitario"] = Final_inventario["Brand"].map(precio_por_brand)

# Valorización total
valor_total_inventario = (
    Final_inventario["onHand"] * Final_inventario["CostoUnitario"]
).sum()

print("Valor total de inventario actual (USD): {:,.2f}".format(valor_total_inventario))
```

✓ 0.0s

```
Valor total de inventario actual (USD): 55,429,569.00
```

Este contraste entre volumen vendido y volumen en inventario ya permite observar que el capital invertido en stock es significativo y justifica la necesidad de optimización.

Bloque 2 – Demanda Implícita por Variación de Inventario

En este bloque se reconstruyó la demanda anual utilizando la variación de inventario, ya que no se trabajó directamente con ventas para estimar consumo estructural.
Se aplicó la lógica contable:

Consumo = Stock Inicial + Compras – Stock Final

Esta fórmula permite estimar cuántas unidades realmente salieron del sistema durante el período.

1- Consolidación de datos

Primero se resumió el stock inicial por producto y sucursal, agrupando el inventario disponible al inicio del período.
Luego se consolidaron las compras del año, sumando la cantidad adquirida por producto y sucursal.
Posteriormente se resumió el stock final al cierre del período.
Finalmente, se unificaron las tres tablas en una sola estructura (tabla “balance”), asegurando que cualquier valor faltante se interpretara como cero para mantener coherencia contable.

```python
# Unir todo en 1 sola tabla (la “tabla balance”)
balance = (
    stock_ini
    .merge(compras, on=["Store","Brand"], how="outer")
    .merge(stock_fin, on=["Store","Brand"], how="outer")
)

# Si falta alguno, se asume 0 (no hubo stock/compras/stock final)
balance[["StockInicial","Compras","StockFinal"]] = balance[["StockInicial","Compras","StockFinal"]].fillna(0)

balance.head()
```

✓ 0.3s

	Store	Brand	StockInicial	Compras	StockFinal
0	1	58	8.0	105.0	11.0
1	1	60	7.0	36.0	0.0
2	1	62	6.0	116.0	7.0
3	1	63	3.0	59.0	7.0
4	1	72	6.0	0.0	4.0

2- Cálculo del Consumo Implícito

Una vez construida la tabla balance, se calculó el consumo implícito para cada combinación producto–sucursal.

Se realizó un control básico de calidad para identificar:

- Consumos positivos
- Consumos cero
- Consumos negativos

Resultados obtenidos:

- **Consumo > 0:** 265.958 registros
- **Consumo = 0:** 8.691 registros
- **Consumo < 0:** 0 registros

La ausencia de consumos negativos indica consistencia estructural en los movimientos de inventario.

```
# Control de calidad básico (consumo negativo / cero / positivo)
negativos = (balance["Consumo"] < 0).sum()
ceros     = (balance["Consumo"] == 0).sum()
positivos = (balance["Consumo"] > 0).sum()

print("Resumen de Consumo:")
print("Consumo > 0 :", positivos)
print("Consumo = 0 :", ceros)
print("Consumo < 0 :", negativos)
```

✓ 0.0s

```
Resumen de Consumo:
Consumo > 0 : 265958
Consumo = 0 : 8691
Consumo < 0 : 0
```

3- Análisis agregado por producto (empresa completa)

Se consolidó el consumo a nivel producto para obtener una visión global de la demanda anual de cada SKU.

Se identificaron:

- **Top 20 productos con mayor consumo**
- **Bottom 20 productos con menor consumo (consumo cero)**

Esto permitió observar una fuerte concentración de demanda en un grupo reducido de productos, mientras que una parte del portafolio no presentó rotación durante el período

```
# Top 20 y Bottom 20
top_20 = consumo_por_producto.head(20)
bottom_20 = consumo_por_producto.tail(20)

print("TOP 20 productos por Consumo:")
display(top_20)

print("BOTTOM 20 productos por Consumo:")
display(bottom_20)
```

✓ 0.0s

```
TOP 20 productos por Consumo:
```

	Brand	Consumo
3378	8111	334939.0
844	1892	253371.0
2568	5111	241963.0

```
BOTTOM 20 productos por Consumo:
```

	Brand	Consumo
6221	19980	0.0
6011	19548	0.0
6981	21856	0.0
10579	38701	0.0
5926	19313	0.0
7731	23568	0.0
11178	44339	0.0
9557	27275	0.0

4- Análisis por sucursal

También se analizaron los productos de mayor consumo dentro de cada sucursal, lo que permitió detectar diferencias en comportamiento entre tiendas y posibles patrones locales de demanda.

```
# Top por sucursal (sin explotar el notebook)
top_por_sucursal = (
    balance[balance["Consumo"] > 0]
    .sort_values(["Store","Consumo"], ascending=[True,False])
    .groupby("Store")
    .head(10)
)

top_por_sucursal.head(30)
```

✓ 0.1s

	Store	Brand	StockInicial	Compras	StockFinal	Consumo
2067	1	8111	58.0	6522.0	417.0	6163.0
376	1	1892	92.0	5318.0	496.0	4914.0
1047	1	3609	21.0	4960.0	378.0	4603.0
1174	1	3837	229.0	4228.0	169.0	4288.0
1290	1	4157	85.0	4414.0	305.0	4194.0
1044	1	3606	52.0	4357.0	220.0	4189.0
1282	1	4135	122.0	4056.0	356.0	3822.0
1871	1	6570	0.0	3630.0	304.0	3326.0
911	1	3315	0.0	3544.0	391.0	3153.0
2291	1	8910	95.0	3139.0	187.0	3047.0
5235	2	3606	135.0	6496.0	218.0	6413.0
6188	2	8068	215.0	3733.0	210.0	3738.0

Cierre del bloque

Este bloque permite reconstruir la demanda real del sistema sin depender exclusivamente del registro de ventas.
Además:

- Se confirmó consistencia contable (sin consumos negativos).
- Se identificó una porción significativa del portafolio sin rotación.
- Se evidenció concentración de consumo en ciertos productos.

Estos resultados constituyen la base técnica para los siguientes bloques:
Clasificación ABC, rotación y análisis de salud del inventario.

Bloque 3 – Clasificación ABC

Objetivo: Priorizar la gestión de inventario según impacto económico real.

1- Construcción del revenue implícito

Se unificó el consumo estimado anual por producto (obtenido en el Bloque 2) con un precio representativo por marca (mediana histórica de compra).

Se calculó:

Revenue Implı´cito=Consumo×Precio RepresentativoRevenue\ Implícito = Consumo \times Precio\ RepresentativoRevenue Implı´cito=Consumo×Precio Representativo

Este valor permite estimar el impacto económico anual de cada SKU, incluso en ausencia de ventas completas consolidadas.

Se eliminaron posibles valores nulos derivados de productos sin precio asignado.

Resultado:
Tabla consolidada con:

- Brand
- Consumo
- PrecioRepresentativo
- RevenueImplícito

```python
# Unir consumo con precio representativo
abc = consumo_por_producto.merge(
    precio_por_brand,
    on="Brand",
    how="left"
)

# Renombrar para claridad
abc = abc.rename(columns={"PurchasePrice": "PrecioRepresentativo"})

# Calcular revenue implícito
abc["RevenueImplicito"] = abc["Consumo"] * abc["PrecioRepresentativo"]

# Eliminar posibles NaN
abc = abc.dropna(subset=["RevenueImplicito"])

abc.head()
```

✓ 0.0s

	Brand	Consumo	PrecioRepresentativo	RevenueImplicito
0	8111	334939.0	0.77	257903.03
1	1892	253371.0	0.72	182427.12
2	5111	241963.0	0.72	174213.36
3	3606	225741.0	0.74	167048.34
4	6570	214238.0	6.77	1450391.26

2- Ordenamiento por impacto económico

Los productos fueron ordenados de mayor a menor según su revenue implícito.

Esto permitió visualizar qué SKUs sostienen económicamente el negocio y preparar la base para el análisis de concentración.

```
# Ordenar por impacto económico
abc = abc.sort_values("RevenueImplicito", ascending=False).reset_index(drop=True)
abc.head()
```

✓ 0.0s

	Brand	Consumo	PrecioRepresentativo	RevenueImplicito
0	1233	142049.0	26.27	3731627.23
1	3405	160247.0	23.19	3716127.93
2	8068	187140.0	18.24	3413433.60
3	4261	200412.0	16.17	3240662.04
4	3545	135838.0	21.89	2973493.82

3- Cálculo de participación y acumulado

Se calculó:

- Participación individual sobre el revenue total.
- Porcentaje acumulado.

Esto permitió medir cómo se concentra el ingreso a medida que se agregan productos en orden descendente de impacto.

Revenue total estimado del período:

$313,385,299.94

Total de productos analizados:

11.503

```
print("Total productos:", len(abc))
print("Total revenue implícito: ${:,.2f}".format(total_revenue))
```

✓ 0.0s

```
Total productos: 11503
Total revenue implícito: $313,385,299.94
```

4- Clasificación ABC

Se aplicó la segmentación clásica de Pareto:

- **A:** hasta 80% del revenue acumulado

- **B:** hasta 95%
- **C:** restante 5%

Resultado obtenido:

```python
total_revenue = abc["RevenueImplicito"].sum()

resumen = (abc.groupby("CategoriaABC")
            .agg(Productos=("Brand","count"),
                 Revenue_USD=("RevenueImplicito","sum"),
                 Consumo_Total=("Consumo","sum"))
            .assign(Pct_Revenue=lambda d: d["Revenue_USD"] / total_revenue)
            .sort_index())

# Formato lindo (solo para mostrar)
resumen_style = resumen.style.format({
    "Revenue_USD": "${:,.2f}",
    "Pct_Revenue": "{:.2%}",
    "Consumo_Total": "{:,.0f}"
})

resumen_style
```

✓ 0.1s

CategoriaABC	Productos	Revenue_USD	Consumo_Total	Pct_Revenue
A	1550	$250,674,054.60	24,520,145	79.99%
B	2097	$47,041,200.60	6,470,548	15.01%
C	7856	$15,670,044.74	1,927,183	5.00%

5- Interpretación del resultado

Se observa una fuerte concentración económica:

- El 13% del portafolio (Categoría A) sostiene el 80% del negocio.
- Más del 68% de los productos (Categoría C) representan solo el 5% del revenue.

Esto indica:

- Alta dependencia de un grupo reducido de SKUs.
- Amplio catálogo de bajo impacto económico.
- Potencial oportunidad de optimización en la gestión de productos C.

6- Visualización – Curva de Pareto

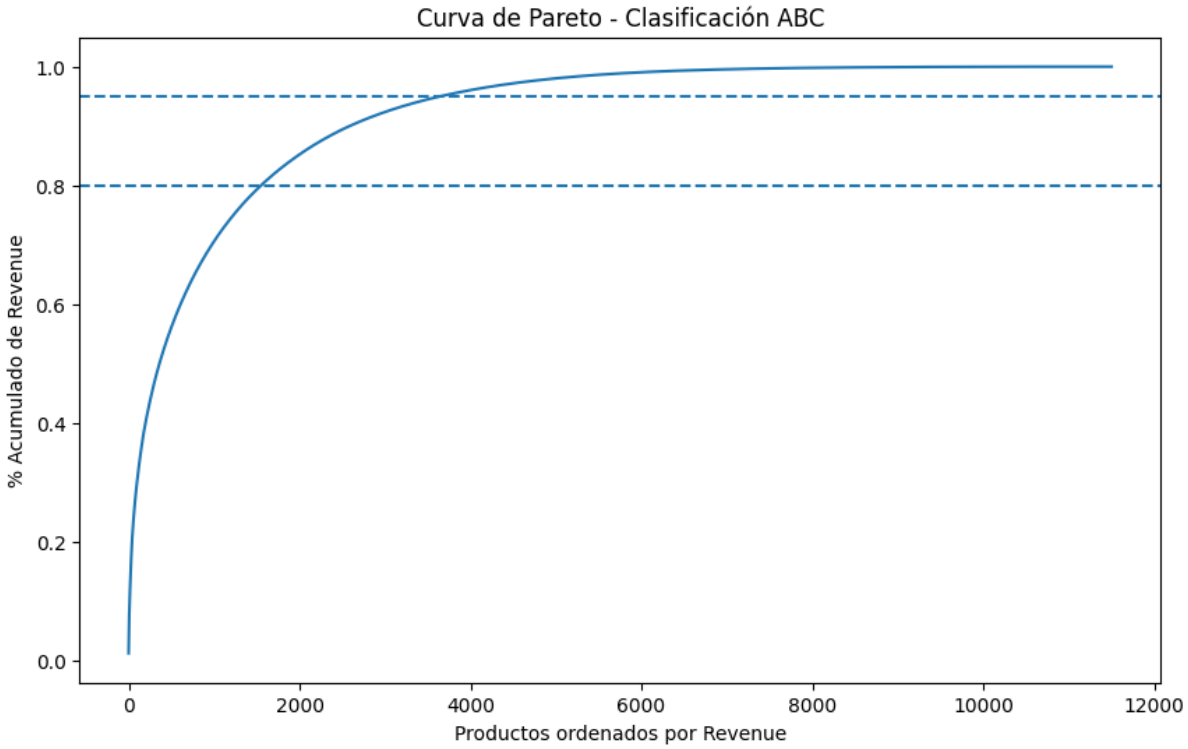
La curva de Pareto confirma visualmente la concentración:

- Fuerte pendiente inicial → alta concentración.
- Zona plana final → muchos productos con impacto marginal.

La clasificación ABC no solo segmenta el portafolio, sino que permite definir distintos niveles de control operativo:

- Categoría A → control estricto y seguimiento frecuente.
- Categoría B → monitoreo intermedio.
- Categoría C → revisión estratégica y análisis de racionalización.

```
plt.figure(figsize=(10,6))
plt.plot(abc["Pct_Acumulado"].values)
plt.axhline(0.80, linestyle="--")
plt.axhline(0.95, linestyle="--")
plt.title("Curva de Pareto - Clasificación ABC")
plt.xlabel("Productos ordenados por Revenue")
plt.ylabel("% Acumulado de Revenue")
plt.show()
```



Cierre del bloque

El análisis ABC evidencia que el negocio presenta una estructura altamente concentrada. Una proporción reducida de productos genera la mayor parte del impacto económico, mientras que la mayoría del catálogo tiene contribución marginal.

Este resultado establece la base para:

- Análisis de rotación (Bloque 4)
- Evaluación de capital inmovilizado
- Revisión estratégica del portafolio

Bloque 4 – Análisis de Rotación de Inventario

Objetivo: El objetivo de este bloque es medir la eficiencia del inventario por producto (Brand), evaluando:

- La rotación anual.
- Los días promedio en inventario.
- La distribución de desempeño por categoría ABC.

Este análisis permite identificar productos con alta eficiencia operativa y productos con potencial riesgo de sobrestock.

Parámetros de evaluación

Se definió un stock promedio mínimo de 5 unidades para considerar la rotación interpretable. Para evitar que outliers distorsionen las visualizaciones, se aplicó un recorte visual al percentil 99 de rotación y días. Adicionalmente, puede fijarse un límite máximo de rotación sólo para gráficos.

- **MIN_INV_PROM:** define qué productos tienen rotación “confiable”.
- **CAP_PERCENTIL:** recorta outliers solo para visualizar.
- **MAX_ROT_PLOT:** recorte fijo para que el gráfico sea legible.

```
# Definir parámetros operativos del análisis
MIN_INV_PROM = 5
CAP_PERCENTIL = 0.99
MAX_ROT_PLOT = None
✓ 0.0s
```

1- Juntamos todo por producto

Se consolidó la información de la tabla `balance` agrupando por `Brand`, obteniendo para cada producto:

- Stock inicial total
- Stock final total
- Consumo total

Esta etapa permite estructurar el análisis a nivel producto, generando una base resumida necesaria para el cálculo posterior de inventario promedio y rotación.

```python
# Consolidar por producto: stock inicial, stock final y consumo total
rotacion = (
    balance
    .groupby("Brand", as_index=False)
    .agg(
        StockInicial=("StockInicial", "sum"),
        StockFinal=("StockFinal", "sum"),
        Consumo=("Consumo", "sum")
    )
)
rotacion.head()
```

✓ 0.1s

	Brand	StockInicial	StockFinal	Consumo
0	58	281.0	385.0	3446.0
1	60	288.0	146.0	1775.0
2	61	12.0	12.0	312.0
3	62	384.0	459.0	3125.0
4	63	365.0	442.0	2778.0

2- Calculamos el inventario promedio

Se calculó el inventario promedio por producto como la media entre el stock inicial y el stock final del período:

InventarioPromedio = (StockInicial + StockFinal) / 2

Este indicador representa el nivel promedio de stock mantenido durante el período analizado y constituye la base para el cálculo de la rotación de inventario.

```python
# Calcular inventario promedio anual
rotacion["InventarioPromedio"] = (
    rotacion["StockInicial"] + rotacion["StockFinal"]) / 2

rotacion[["Brand","InventarioPromedio"]].head()
```

✓ 0.0s

	Brand	InventarioPromedio
0	58	333.0
1	60	217.0
2	61	12.0
3	62	421.5
4	63	403.5

3- Criterio de Evaluabilidad

Se definió un criterio para determinar qué productos presentan condiciones adecuadas para el cálculo de rotación.

Un producto se considera **evaluable** cuando:

- El inventario promedio es mayor o igual al umbral mínimo definido (`MIN_INV_PROM`).

- El consumo es mayor a cero.

Este control evita distorsiones en la métrica de rotación derivadas de inventarios promedio muy bajos o inexistentes, asegurando que los indicadores calculados sean interpretables y consistentes.

```python
# Marcar productos donde rotación es interpretable (evita explosiones por denominador chico)
rotacion["Evaluable"] = (
    rotacion["InventarioPromedio"] >= MIN_INV_PROM) & (rotacion["Consumo"] > 0)

rotacion.head()
```

✓ 0.0s

	Brand	StockInicial	StockFinal	Consumo	InventarioPromedio	Evaluable
0	58	281.0	385.0	3446.0	333.0	True
1	60	288.0	146.0	1775.0	217.0	True
2	61	12.0	12.0	312.0	12.0	True
3	62	384.0	459.0	3125.0	421.5	True
4	63	365.0	442.0	2778.0	403.5	True

4- Calculamos la rotación

La rotación anual se calculó únicamente para los productos definidos como evaluables, utilizando la siguiente relación:

Rotación = Consumo / InventarioPromedio

Con el fin de facilitar la interpretación ejecutiva, los resultados fueron redondeados y convertidos a valores enteros, eliminando decimales no relevantes para el análisis.

Este indicador refleja la cantidad de veces que el inventario promedio se renovó durante el período evaluado.

```python
# Calcular rotación anual solo para productos evaluables y redondear a entero
rotacion["Rotacion"] = np.where(
    rotacion["Evaluable"],
    (rotacion["Consumo"] / rotacion["InventarioPromedio"]).round(0),
    np.nan
)

# Convertir a entero (sin decimales visibles)
rotacion["Rotacion"] = rotacion["Rotacion"].astype("Int64")

rotacion.head()
```

✓ 0.0s

	Brand	StockInicial	StockFinal	Consumo	InventarioPromedio	Evaluable	Rotacion
0	58	281.0	385.0	3446.0	333.0	True	10
1	60	288.0	146.0	1775.0	217.0	True	8

5- Días promedio en inventario

Para los productos con rotación válida, se calculó el indicador de permanencia promedio en inventario mediante la siguiente relación:

DíasInventario = 365 / Rotación

Este indicador estima la cantidad de días que, en promedio, una unidad permanece en stock antes de ser consumida.
Se aplicaron controles para evitar valores infinitos derivados de divisiones inválidas, y los resultados fueron redondeados y convertidos a formato entero para facilitar su lectura.
Este indicador complementa la rotación, ofreciendo una perspectiva temporal sobre la dinámica del inventario.

```python
# Calcular días promedio en inventario solo si rotación es válida
rotacion["DiasInventario"] = np.where(
    rotacion["Rotacion"].notna() & (rotacion["Rotacion"] > 0),
    365 / rotacion["Rotacion"],
    np.nan
)

# Limpiar infinitos antes de convertir
rotacion.replace([np.inf, -np.inf], np.nan, inplace=True)

# Redondear antes de convertir
rotacion["Rotacion"] = rotacion["Rotacion"].round(0)
rotacion["DiasInventario"] = rotacion["DiasInventario"].round(0)

# Convertir a enteros (manteniendo NaN permitidos)
rotacion["Rotacion"] = rotacion["Rotacion"].astype("Int64")
rotacion["DiasInventario"] = rotacion["DiasInventario"].astype("Int64")

rotacion.head()
```

✓ 0.0s

	Brand	StockInicial	StockFinal	Consumo	InventarioPromedio	Evaluable	Rotacion	DiasInventario
0	58	281.0	385.0	3446.0	333.0	True	10	36
1	60	288.0	146.0	1775.0	217.0	True	8	46

6- Diagnóstico de Evaluabilidad

Se realizó un análisis de alcance para determinar cuántos productos cumplen con los criterios mínimos definidos para el cálculo de rotación.

Se reportan:

- Total de productos analizados.
- Cantidad de productos no evaluables.
- Porcentaje que representan sobre el total.

En el período analizado:

- Total de productos: 11.503
- Productos no evaluables: 2.049 (17,81%)

Estos productos fueron excluidos del cálculo de rotación debido a inventario promedio inferior al umbral definido o consumo igual a cero.

Este diagnóstico permite dimensionar el alcance real del análisis y documentar explícitamente las exclusiones metodológicas aplicadas.

```
# Reportar cuántos productos quedaron fuera por criterio de evaluabilidad
total = len(rotacion)
no_eval = (~rotacion["Evaluable"]).sum()
pct_no_eval = round(no_eval / total * 100, 2)

print("DIAGNÓSTICO BLOQUE 4")
print(f"Total Brands: {total}")
print(f"No evaluables (inv promedio < {MIN_INV_PROM} o consumo=0): {no_eval} ({pct_no_eval}%)")
✓ 0.0s

DIAGNÓSTICO BLOQUE 4
Total Brands: 11503
No evaluables (inv promedio < 5 o consumo=0): 2049 (17.81%)
```

7- Control de Outliers para Visualización

Con el fin de evitar que valores extremos distorsionen la interpretación gráfica, se aplicó un recorte visual basado en el percentil definido (`CAP_PERCENTIL`).

Se calcularon:

- Percentil 99 de la Rotación.
- Percentil 99 de los Días en Inventario.

Estos valores se utilizaron exclusivamente para generar versiones ajustadas de las métricas destinadas a visualización (`Rotacion_plot` y `DiasInventario_plot`), mediante un límite superior (*clipping*).

Es importante destacar que:

- Los valores originales no fueron modificados.
- El recorte se aplica únicamente con fines gráficos.
- La interpretación analítica se basa en los datos reales.

En el período analizado:

- Cap Rotación (p99): 46,79
- Cap Días (p99): 3.477

Este procedimiento mejora la legibilidad de los gráficos sin alterar la integridad del análisis.

```python
# Aplicar cap percentil para que los gráficos no queden aplastados por outliers
if len(rot_eval) > 0:
    cap_rot = rot_eval["Rotacion"].quantile(CAP_PERCENTIL)
    cap_days = rot_eval["DiasInventario"].quantile(CAP_PERCENTIL)

    rot_eval["Rotacion_plot"] = rot_eval["Rotacion"].clip(upper=cap_rot)
    rot_eval["DiasInventario_plot"] = rot_eval["DiasInventario"].clip(upper=cap_days)

    if MAX_ROT_PLOT is not None:
        rot_eval["Rotacion_plot"] = rot_eval["Rotacion_plot"].clip(upper=MAX_ROT_PLOT)

    print(f"Cap Rotación (p{int(CAP_PERCENTIL*100)}): {round(cap_rot,2)}")
    print(f"Cap Días (p{int(CAP_PERCENTIL*100)}): {round(cap_days,1)}")
else:
    print("No hay filas evaluables. Probá bajar MIN_INV_PROM.")
```

✓ 0.0s

```
Cap Rotación (p99): 46.79
Cap Días (p99): 3477.0
```

8- Análisis de Rotación por Categoría ABC

Con el objetivo de evaluar el comportamiento del inventario según su criticidad económica, se realizó un análisis de rotación segmentado por categoría ABC, considerando únicamente los productos evaluables.

Para cada categoría se calcularon:

- Cantidad de productos.
- Rotación promedio y mediana.
- Días promedio y mediana en inventario.

Resultados del período analizado:

- **Categoría A**
    - Productos: 1.541
    - Rotación promedio: 12,42
    - Días promedio en inventario: 58
- **Categoría B**
    - Productos: 2.047
    - Rotación promedio: 10,86
    - Días promedio en inventario: 88,4
- **Categoría C**
    - Productos: 5.866
    - Rotación promedio: 4,05

- Días promedio en inventario: 427,8

Interpretación general:

- Los productos de categoría A presentan mayor dinamismo y menor permanencia en inventario.
- Los productos C muestran rotaciones significativamente más bajas y tiempos de permanencia elevados, lo que indica menor eficiencia operativa y potencial acumulación de stock.
- La categoría B presenta un comportamiento intermedio y coherente con su nivel de criticidad.

Este análisis confirma la consistencia entre la clasificación ABC y la dinámica real del inventario.

```python
# Resumir rotación y días por categoría ABC (solo evaluables)
resumen_rot = (
    rot_eval
    .groupby("CategoriaABC", as_index=False)
    .agg(
        Productos=("Brand", "count"),
        Rotacion_Promedio=("Rotacion", "mean"),
        Rotacion_Mediana=("Rotacion", "median"),
        Dias_Promedio=("DiasInventario", "mean"),
        Dias_Mediana=("DiasInventario", "median"),
    )
)

resumen_rot["Rotacion_Promedio"] = resumen_rot["Rotacion_Promedio"].round(2)
resumen_rot["Rotacion_Mediana"]  = resumen_rot["Rotacion_Mediana"].round(2)
resumen_rot["Dias_Promedio"]     = resumen_rot["Dias_Promedio"].round(1)
resumen_rot["Dias_Mediana"]      = resumen_rot["Dias_Mediana"].round(1)

resumen_rot
```

✓ 0.0s

	CategoriaABC	Productos	Rotacion_Promedio	Rotacion_Mediana	Dias_Promedio	Dias_Mediana
0	A	1541	12.42	7.44	58.0	49.0
1	B	2047	10.86	4.97	88.4	73.4
2	C	5866	4.05	2.22	427.8	164.2

9- Distribución de Rotación Anual

Se elaboró un histograma de la rotación anual utilizando la versión ajustada por percentil 99 (cap p99), con el objetivo de evitar distorsiones visuales provocadas por valores extremos.

El gráfico muestra la frecuencia de productos según su nivel de rotación.

Observaciones principales:

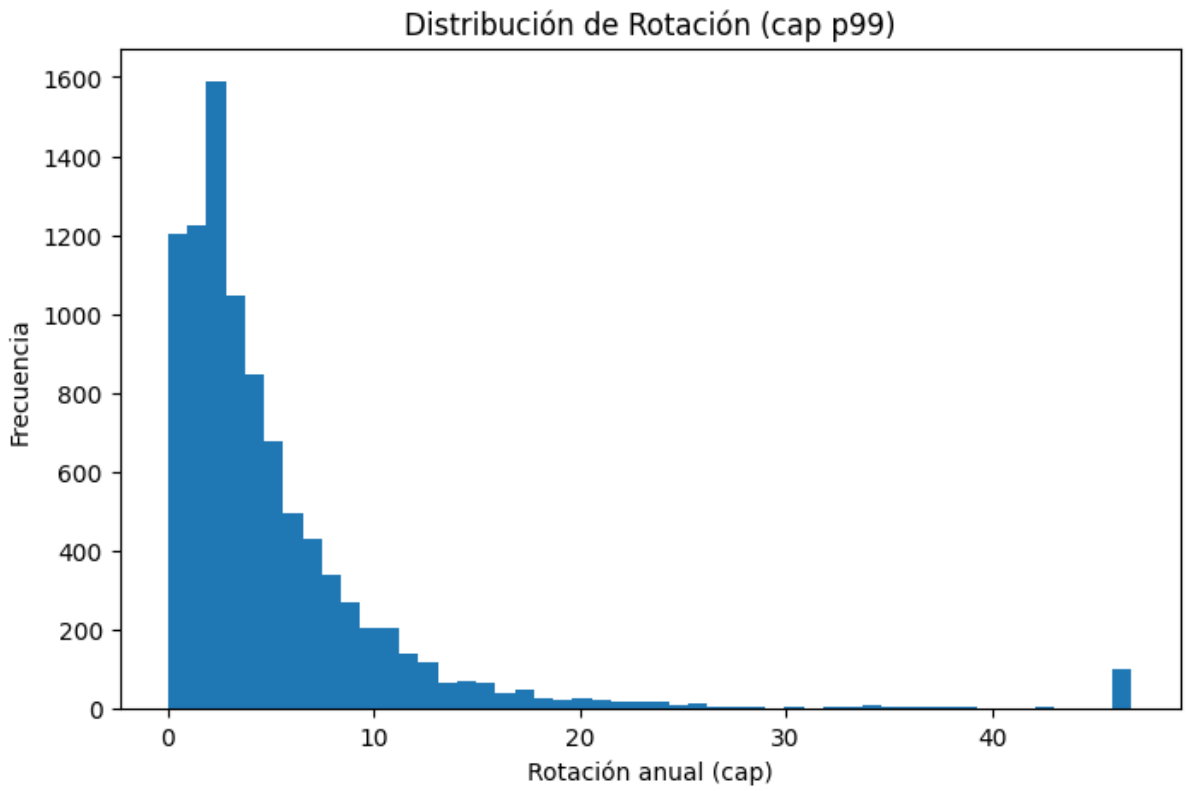
- La mayor concentración de productos se encuentra en rangos de rotación bajos (entre 0 y 10).
- A medida que aumenta la rotación, la frecuencia disminuye progresivamente.
- Existe una cola derecha extensa, lo que indica la presencia de productos con rotaciones significativamente superiores al promedio.

La distribución presenta una asimetría positiva (sesgo hacia la derecha), lo que implica que la mayoría de los productos tiene rotaciones moderadas o bajas, mientras que un grupo reducido presenta niveles elevados de dinamismo.

Este comportamiento es consistente con estructuras de inventario típicas, donde pocos productos concentran alta rotación y una mayoría mantiene ritmos más conservadores.

El uso del cap percentil mejora la legibilidad del gráfico sin alterar los datos originales del análisis.

```
# Visualizar la distribución de rotación evitando outliers extremos
plt.figure(figsize=(8,5))
plt.hist(rot_eval["Rotacion_plot"], bins=50)
plt.title("Distribución de Rotación (cap p99)")
plt.xlabel("Rotación anual (cap)")
plt.ylabel("Frecuencia")
plt.show()
✓ 0.3s
```



10- Relación entre Consumo y Rotación por Categoría ABC

Se elaboró un gráfico de dispersión para analizar la relación entre la demanda (Consumo) y la eficiencia del inventario (Rotación), segmentando los productos por categoría ABC.

- Eje X: Consumo total.
- Eje Y: Rotación anual (versión ajustada por cap p99 para visualización).

- Cada punto representa un producto.
- El color identifica la categoría ABC.

Observaciones principales:

- Los productos de categoría A tienden a concentrarse en zonas de mayor consumo y rotación moderada a alta.
- La categoría B presenta un comportamiento intermedio, con dispersión moderada.
- Los productos de categoría C se concentran mayormente en niveles bajos de consumo y baja rotación.

Se observa una relación general positiva: a mayores niveles de consumo, suele corresponder una mayor rotación, aunque con variabilidad significativa entre productos.

Interpretación:

El gráfico permite identificar distintos perfiles operativos:

- Productos de alto consumo y alta rotación: inventario eficiente y dinámico.
- Productos de bajo consumo y baja rotación: potencial acumulación de stock.
- Productos de alto consumo pero rotación moderada: posible oportunidad de optimización del nivel de inventario.

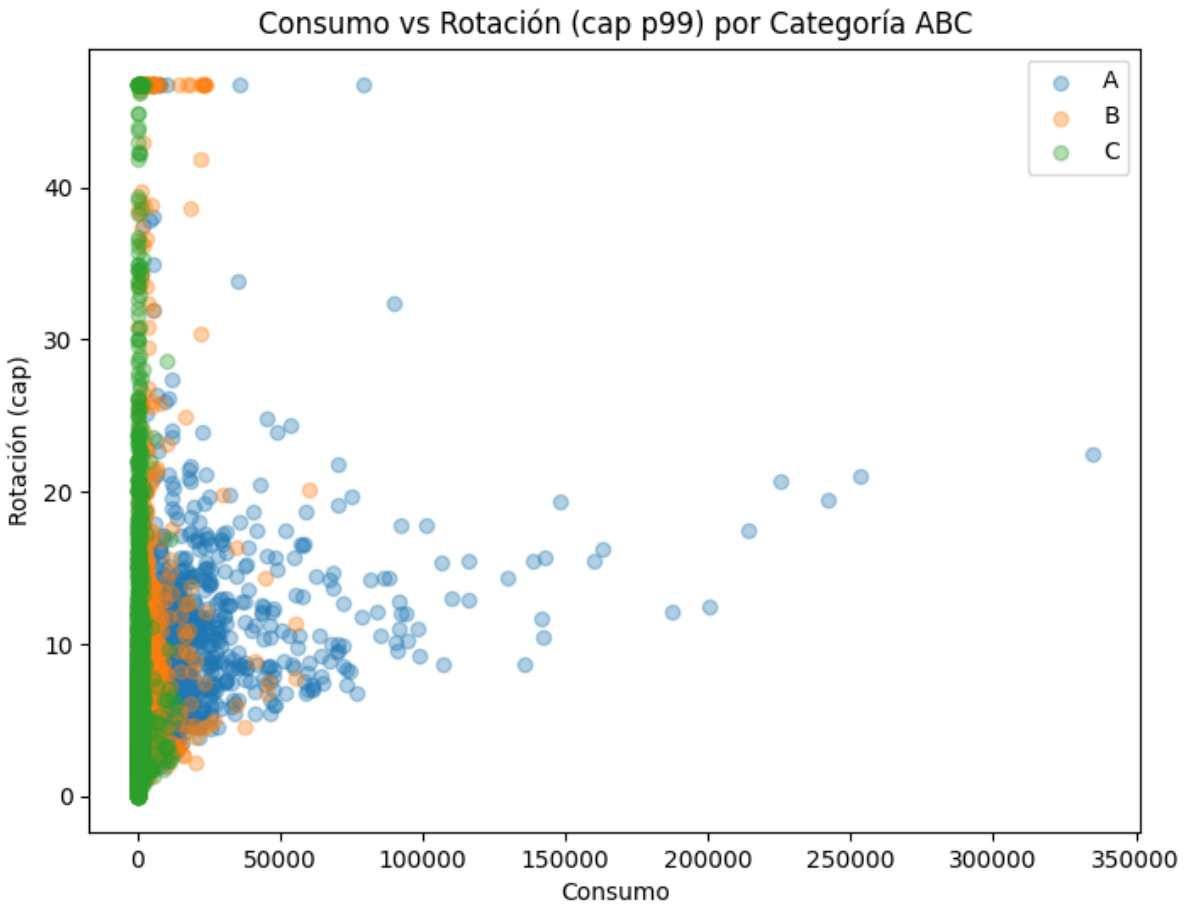
Este análisis complementa el estudio cuantitativo y facilita la identificación visual de patrones operativos dentro de cada categoría.

```python
# Relacionar demanda (Consumo) vs eficiencia (Rotación) segmentando por ABC
plt.figure(figsize=(8,6))

cats = ["A", "B", "C"]
for cat in cats:
    sub = rot_eval[rot_eval["CategoriaABC"] == cat]
    if len(sub) > 0:
        plt.scatter(sub["Consumo"], sub["Rotacion_plot"], alpha=0.35, label=cat)

sub_na = rot_eval[~rot_eval["CategoriaABC"].isin(cats)]
if len(sub_na) > 0:
    plt.scatter(sub_na["Consumo"], sub_na["Rotacion_plot"], alpha=0.20, label="SinABC")

plt.title("Consumo vs Rotación (cap p99) por Categoría ABC")
plt.xlabel("Consumo")
plt.ylabel("Rotación (cap)")
plt.legend()
plt.show()
```
✓ 1.2s



11. Identificación de Productos Extremos (Rotación Real)

Con el objetivo de detectar comportamientos extremos dentro del inventario, se identificaron los 10 productos con mayor rotación y los 10 con menor rotación, utilizando los valores reales (sin aplicar recortes visuales).

Top 10 Rotación

Los productos con mayor rotación presentan:

- Inventarios promedio relativamente bajos.
- Niveles de consumo elevados en relación con el stock disponible.
- Días promedio en inventario extremadamente bajos.

Estos casos reflejan productos de alta dinámica, donde el inventario se renueva con gran frecuencia durante el período analizado.

Bottom 10 Rotación

Los productos con menor rotación presentan:

- Inventarios promedio elevados en relación con su consumo.
- Valores de rotación cercanos a cero.

- Días promedio en inventario significativamente altos.

Este comportamiento indica productos de baja dinámica operativa, con alta permanencia en stock y potencial riesgo de acumulación o sobrestock.

Interpretación General

El análisis de extremos permite:

- Detectar productos altamente eficientes en términos de renovación de inventario.
- Identificar productos con baja rotación que pueden requerir revisión de políticas de compra, reposición o continuidad comercial.

Este apartado complementa el análisis estadístico general, proporcionando una visión puntual sobre los casos más representativos del comportamiento del inventario.

```python
# Identificar productos extremos usando rotación real (sin cap)
top10 = (rot_eval.sort_values("Rotacion", ascending=False)
         .head(10)[["Brand","Consumo","InventarioPromedio","Rotacion","DiasInventario","CategoriaABC"]])

bottom10 = (rot_eval.sort_values("Rotacion", ascending=True)
            .head(10)[["Brand","Consumo","InventarioPromedio","Rotacion","DiasInventario","CategoriaABC"]])

print("TOP 10 Rotación (evaluables):")
display(top10)

print("BOTTOM 10 Rotación (evaluables):")
display(bottom10)
```

✓ 0.0s

```
TOP 10 Rotación (evaluables):
```

	Brand	Consumo	InventarioPromedio	Rotacion	DiasInventario	CategoriaABC
2272	4260	79042.0	19.0	4160.11	0.1	A
8579	25202	5547.0	6.0	924.50	0.4	A
6960	21791	5560.0	7.5	741.33	0.5	B
614	1195	14677.0	22.0	667.14	0.5	B
8791	25587	3344.0	6.0	557.33	0.7	B
1675	3288	4669.0	9.5	491.47	0.7	A
559	1076	17309.0	35.5	487.58	0.7	B
1968	3737	2575.0	5.5	468.18	0.8	B
647	1261	23667.0	53.5	442.37	0.8	B
8940	25827	2647.0	6.5	407.23	0.9	B

```
BOTTOM 10 Rotación (evaluables):
```

	Brand	Consumo	InventarioPromedio	Rotacion	DiasInventario	CategoriaABC
9583	27342	3.0	531.0	0.01	64605.0	C
9556	27274	1.0	86.5	0.01	31572.5	C
9656	27553	1.0	113.5	0.01	41427.5	C
7244	22591	1.0	72.5	0.01	26462.5	C
9767	27969	1.0	83.5	0.01	30477.5	C
4867	15880	2.0	299.0	0.01	54567.5	C
4310	13557	1.0	60.5	0.02	22082.5	C
943	2094	2.0	113.0	0.02	20622.5	C
4874	15901	1.0	59.5	0.02	21717.5	C
4478	14219	1.0	29.5	0.03	10767.5	C

Cierre del bloque

El análisis de rotación permitió evaluar la eficiencia operativa del inventario a nivel producto, identificando diferencias significativas en la dinámica de renovación del stock.

Se determinó que:

- La mayoría de los productos presenta rotaciones moderadas o bajas, concentrándose en los primeros rangos de la distribución.
- Las categorías A muestran mayor dinamismo y menor permanencia promedio en inventario, lo cual es consistente con su mayor relevancia económica.
- Las categorías C presentan rotaciones considerablemente más bajas y tiempos prolongados de permanencia, evidenciando menor eficiencia operativa.
- Un 17,81% de los productos no resultó evaluable bajo los criterios definidos, lo que fue debidamente documentado para garantizar transparencia metodológica.

El análisis de extremos confirmó la coexistencia de productos altamente dinámicos con otros de muy baja rotación, lo que sugiere oportunidades de optimización en la gestión de inventario, particularmente en los segmentos de baja demanda.

En conjunto, el Bloque 4 proporciona una visión estructural del comportamiento del inventario, estableciendo una base sólida para el análisis posterior de sobrestock, quiebres y oportunidades de mejora en el Bloque 5.

Bloque 5 – Salud del Inventario

Objetivo: Detectar excesos y riesgos operativos en el inventario, identificando posibles pérdidas de ventas y capital inmovilizado.

Respondemos preguntas más estratégicas:

- ¿Estamos perdiendo ventas por falta de stock?
- ¿Tenemos capital inmovilizado en productos que no se mueven?
- ¿El inventario está bien distribuido según la importancia económica (ABC)?
- ¿Las tiendas están equilibradas entre demanda y disponibilidad?

Este bloque traduce los números en riesgo operativo y financiero real.

Parámetros generales

Se definieron criterios cuantitativos para establecer umbrales de análisis y asegurar consistencia en la identificación de riesgos y desbalances del inventario.

Estos parámetros determinan:

- Qué productos se consideran de **alto o bajo stock**.
- Qué productos se consideran de **alta o baja demanda**.
- Cómo se segmentan las tiendas en la matriz operativa 2x2.
- Cuántos casos extremos se presentan en los listados finales.

Criterios establecidos

- **Stock alto:** percentil 90
- **Stock bajo:** percentil 25
- **Consumo alto:** percentil 75
- **Consumo bajo:** percentil 25

Estos cortes permiten identificar extremos relativos dentro del propio universo analizado, evitando el uso de valores absolutos que podrían distorsionar la lectura.

Segmentación de tiendas

Para la clasificación 2x2 (DemandaAlta/StockAlto, etc.), se utilizó como criterio de corte la **mediana** del consumo y del stock total por tienda.

Este enfoque permite una división balanceada del universo, facilitando la comparación entre grupos.

```python
# Parámetros generales

TOP_N = 10

# Umbrales para productos (percentiles sobre Brand)
PCT_STOCK_ALTO = 0.90
PCT_STOCK_BAJO = 0.25
PCT_CONSUMO_ALTO = 0.75
PCT_CONSUMO_BAJO = 0.25

# Corte para recategorización de tiendas
CORTE_TIENDA = "mediana"   # "mediana" (simple) o "p75_p25"

print(" PARÁMETROS")
print(f"TOP_N productos a mostrar: {TOP_N}")
print(f"Stock alto: p{int(PCT_STOCK_ALTO*100)} | Stock bajo: p{int(PCT_STOCK_BAJO*100)}")
print(f"Consumo alto: p{int(PCT_CONSUMO_ALTO*100)} | Consumo bajo: p{int(PCT_CONSUMO_BAJO*100)}")
print(f"Corte para tiendas (2x2): {CORTE_TIENDA}")
```

✓ 0.0s

```
 PARÁMETROS
TOP_N productos a mostrar: 10
Stock alto: p90 | Stock bajo: p25
Consumo alto: p75 | Consumo bajo: p25
Corte para tiendas (2x2): mediana
```

1- Consolidación por producto

Se centralizó la información a nivel **Brand**, sumando el stock final y el consumo total de cada producto.
Esto permite analizar el inventario desde una visión consolidada, evitando distorsiones por sucursal o movimiento individual.

Se integró la clasificación **ABC** para segmentar los productos según su relevancia económica dentro del negocio.

Además, se incorporaron dos indicadores operativos:

- **TieneDemanda:** consumo mayor a cero.
  StockCero: inventario final igual a cero.

Panorama del período

- **Total de productos:** 11.503
- **Productos con demanda activa:** 11.237
- **Productos con stock final en cero:** 2.004

La distribución ABC mantiene una estructura consistente, con mayor concentración en categoría C y menor volumen en categoría A.

```python
# Centralizamos las métricas de inventario y demanda a nivel de producto, integrando la clasificación estratégica ABC para segmentar el estado operativo del stock.

inv_brand = (
    balance
    .groupby("Brand", as_index=False)
    .agg(
        StockFinal=("StockFinal", "sum"),
        Consumo=("Consumo", "sum")
    )
)

# Unir ABC
inv_brand = inv_brand.merge(abc[["Brand", "CategoriaABC"]], on="Brand", how="left")
inv_brand["CategoriaABC"] = inv_brand["CategoriaABC"].fillna("SinABC")

# Métricas útiles
inv_brand["TieneDemanda"] = inv_brand["Consumo"] > 0
inv_brand["StockCero"] = inv_brand["StockFinal"] == 0

print("Marcas totales:", len(inv_brand))
print("Marcas con demanda (Consumo>0):", int(inv_brand["TieneDemanda"].sum()))
print("Marcas con stock final = 0:", int(inv_brand["StockCero"].sum()))
print("Distribución ABC (conteo):")
print(inv_brand["CategoriaABC"].value_counts(dropna=False))
```

✓ 0.1s

```
Marcas totales: 11503
Marcas con demanda (Consumo>0): 11237
Marcas con stock final = 0: 2004
Distribución ABC (conteo):
CategoriaABC
C    7856
B    2097
A    1550
Name: count, dtype: int64
```

2- Identificación de quiebre operativo

(Stock final = 0 y consumo > 0)

Se identificaron productos con demanda activa pero sin inventario disponible al cierre del período.
Este escenario representa pérdida potencial inmediata de ventas.

- **Productos en quiebre:** 1.939 (16,86% del total)
- **Consumo asociado:** 1,12% del consumo total

Distribución por categoría ABC

- C: 96,24%
- B: 3,40%
- A: 0,36%

El quiebre afecta a una proporción relevante del catálogo, aunque el impacto económico es acotado, ya que se concentra mayormente en productos de categoría C.

La baja incidencia en categoría A indica que los productos estratégicos presentan un nivel de cobertura razonable.

```python
# Identificamo productos con demanda activa pero sin inventario disponible, señalando una pérdida potencial de ventas inmediata.

quiebre = inv_brand[(inv_brand["StockCero"]) & (inv_brand["TieneDemanda"])].copy()

pct_brands_quiebre = round(len(quiebre) / len(inv_brand) * 100, 2)
consumo_total = inv_brand["Consumo"].sum()
consumo_quiebre = quiebre["Consumo"].sum()
pct_consumo_en_quiebre = round(consumo_quiebre / consumo_total * 100, 2) if consumo_total > 0 else 0

print("QUIEBRE (StockFinal=0 y Consumo>0)")
print(f"Brands en quiebre: {len(quiebre)} / {len(inv_brand)} ({pct_brands_quiebre}%)")
print(f"Consumo asociado al quiebre: {consumo_quiebre:,.0f} / {consumo_total:,.0f} ({pct_consumo_en_quiebre}%)")

print("ABC dentro del quiebre (conteo y %):")
mix_q = (quiebre["CategoriaABC"].value_counts(dropna=False).to_frame("Cantidad"))
mix_q["%"] = (mix_q["Cantidad"] / len(quiebre) * 100).round(2)
display(mix_q)

print("TOP productos en quiebre (más consumo):")
display(quiebre.sort_values("Consumo", ascending=False).head(TOP_N)[
    ["Brand","CategoriaABC","Consumo","StockFinal"]
])
```

✓ 0.0s

```
QUIEBRE (StockFinal=0 y Consumo>0)
Brands en quiebre: 1939 / 11503 (16.86%)
Consumo asociado al quiebre: 370,174 / 32,917,876 (1.12%)
ABC dentro del quiebre (conteo y %):
```

CategoriaABC	Cantidad	%
C	1866	96.24
B	66	3.40
A	7	0.36

```
TOP productos en quiebre (más consumo):
```

	Brand	CategoriaABC	Consumo	StockFinal
725	1476	A	15044.0	0.0
3587	8606	C	10465.0	0.0
8554	25163	B	10367.0	0.0
3471	8358	A	8065.0	0.0
6183	19896	A	7426.0	0.0
2693	5390	C	7007.0	0.0
8455	24995	B	5661.0	0.0
8728	25504	B	5531.0	0.0

3- Identificación de sobrestock

(Stock alto y consumo bajo)

Se identificaron productos con inventario ubicado en el percentil superior (p90) y consumo en el percentil inferior (p25).
Este cruce permite detectar capital inmovilizado en productos de baja rotación.

- **Productos en sobrestock:** 1 (0,01% del total)
  Stock asociado: 0,02% del inventario total

Distribución por categoría ABC

- 100% categoría C

No se observa un problema estructural de sobreinventario.
El único caso detectado pertenece a categoría C, lo que indica bajo impacto económico.

En términos generales, el nivel de stock acumulado es consistente con el comportamiento de la demanda.

```python
# Detectamos productos con inventario excesivo y baja rotación mediante cuantiles, localizando el capital inmovilizado que genera ineficiencia financiera.

stock_alto = inv_brand["StockFinal"].quantile(PCT_STOCK_ALTO)
consumo_bajo = inv_brand["Consumo"].quantile(PCT_CONSUMO_BAJO)

sobrestock = inv_brand[
    (inv_brand["StockFinal"] >= stock_alto) &
    (inv_brand["Consumo"] <= consumo_bajo)
].copy()

pct_brands_sobrestock = round(len(sobrestock) / len(inv_brand) * 100, 2)
stock_total = inv_brand["StockFinal"].sum()
stock_sobrestock = sobrestock["StockFinal"].sum()
pct_stock_inmov = round(stock_sobrestock / stock_total * 100, 2) if stock_total > 0 else 0

print("SOBREESTOCK (Stock alto + Consumo bajo)")
print(f"Umbral Stock alto (%{int(PCT_STOCK_ALTO*100)}): {stock_alto:,.2f}")
print(f"Umbral Consumo bajo (%{int(PCT_CONSUMO_BAJO*100)}): {consumo_bajo:,.2f}")
print(f"Marcas en sobrestock: {len(sobrestock)} / {len(inv_brand)} ({pct_brands_sobrestock}%)")
print(f"Stock asociado al sobrestock: {stock_sobrestock:,.0f} / {stock_total:,.0f} ({pct_stock_inmov}%)")

print(" ABC dentro del sobrestock (conteo y %):")
mix_s = (sobrestock["CategoriaABC"].value_counts(dropna=False).to_frame("Cantidad"))
mix_s["%"] = (mix_s["Cantidad"] / len(sobrestock) * 100).round(2)
display(mix_s)

print("TOP productos en sobrestock (más stock):")
display(sobrestock.sort_values("StockFinal", ascending=False).head(TOP_N)[
    ["Brand","CategoriaABC","StockFinal","Consumo"]
])
✓ 0.0s
```

```
SOBREESTOCK (Stock alto + Consumo bajo)
Umbral Stock alto (%90): 1,097.00
Umbral Consumo bajo (%25): 23.00
Marcas en sobrestock: 1 / 11503 (0.01%)
Stock asociado al sobrestock: 1,140 / 4,885,776 (0.02%)
 ABC dentro del sobrestock (conteo y %):
```

	Cantidad	%
CategoriaABC		
C	1	100.0

```
TOP productos en sobrestock (más stock):
```

	Brand	CategoriaABC	StockFinal	Consumo
555	1071	C	1140.0	0.0

4- Identificación de riesgo crítico

(Stock final = 0 y consumo alto)

Se identificaron productos con inventario agotado cuyo nivel de demanda se encuentra en el cuartil superior (top 25%).
A diferencia del quiebre general, aquí se priorizan los casos que más impactan en facturación.

- **Productos en riesgo crítico:** 52 (0,45% del total)
- **Consumo asociado:** 0,63% del consumo total

Distribución por categoría ABC

- B: 69,23%
- C: 17,31%

- A: 13,46%

El riesgo crítico es bajo en volumen, pero cualitativamente relevante.
A diferencia del quiebre general (que se concentraba en categoría C), aquí el riesgo se distribuye principalmente en categorías B y A, lo que implica mayor sensibilidad económica.

Si bien el porcentaje total afectado es reducido, estos casos deben monitorearse de forma prioritaria por su potencial impacto en ingresos.

```python
# Filtramos los productos de alta rotación (Top 25% de demanda) que tienen stock agotado, priorizando las rupturas de inventario que más impactan en la facturación.

consumo_alto = inv_brand["Consumo"].quantile(PCT_CONSUMO_ALTO)

riesgo_critico = inv_brand[
    (inv_brand["StockFinal"] == 0) &
    (inv_brand["Consumo"] >= consumo_alto)
].copy()

pct_brands_riesgo = round(len(riesgo_critico) / len(inv_brand) * 100, 2)
consumo_riesgo = riesgo_critico["Consumo"].sum()
pct_consumo_riesgo = round(consumo_riesgo / consumo_total * 100, 2) if consumo_total > 0 else 0

print("RIESGO CRÍTICO (Stock=0 + Consumo alto)")
print(f"Umbral Consumo alto (%{int(PCT_CONSUMO_ALTO*100)}): {consumo_alto:,.2f}")
print(f"Brands críticos: {len(riesgo_critico)} / {len(inv_brand)} ({pct_brands_riesgo}%)")
print(f"Consumo asociado a críticos: {consumo_riesgo:,.0f} / {consumo_total:,.0f} ({pct_consumo_riesgo}%)")

print("ABC dentro del riesgo crítico (conteo y %):")
mix_rc = (riesgo_critico["CategoriaABC"].value_counts(dropna=False).to_frame("Cantidad"))
mix_rc["%"] = (mix_rc["Cantidad"] / len(riesgo_critico) * 100).round(2)
display(mix_rc)

print("TOP críticos (más consumo):")
display(riesgo_critico.sort_values("Consumo", ascending=False).head(TOP_N)[
    ["Brand","CategoriaABC","Consumo","StockFinal"]
])
✓ 0.0s
```

```
RIESGO CRÍTICO (Stock=0 + Consumo alto)
Umbral Consumo alto (%75): 1,643.00
Brands críticos: 52 / 11503 (0.45%)
Consumo asociado a críticos: 205,840 / 32,917,876 (0.63%)
ABC dentro del riesgo crítico (conteo y %):
```

	Cantidad	%
CategoriaABC		
B	36	69.23
C	9	17.31
A	7	13.46

```
TOP críticos (más consumo):
```

	Brand	CategoriaABC	Consumo	StockFinal
725	1476	A	15044.0	0.0
3587	8606	C	10465.0	0.0
8554	25163	B	10367.0	0.0
3471	8358	A	8065.0	0.0
6183	19896	A	7426.0	0.0
2693	5390	C	7007.0	0.0
8455	24995	B	5661.0	0.0
8728	25504	B	5531.0	0.0
8785	25577	A	5442.0	0.0
8122	24369	A	5322.0	0.0

5- Segmentación de tiendas en 4 cuadrantes (demanda vs stock)

Se consolidaron las métricas por **Store** (StockFinal y Consumo) para clasificar cada tienda según su perfil operativo y detectar desbalances de abastecimiento.

Criterio de corte

Se definieron umbrales para separar **alto/bajo** en ambas variables:

- **Stock**: corte sobre `StockFinal`
- **Demanda**: corte sobre `Consumo`

El corte se calcula según el parámetro `CORTE_TIENDA` (mediana o percentiles), y se utiliza para categorizar de forma consistente a todas las tiendas.

Clasificación por niveles

A cada tienda se le asignan dos etiquetas:

- **NivelStock:** StockAlto / StockBajo
- **NivelDemanda:** DemandaAlta / DemandaBaja

Con estas etiquetas se genera el **TipoTienda**, formando los 4 cuadrantes:

- DemandaAlta + StockAlto
- DemandaAlta + StockBajo
- DemandaBaja + StockAlto
- DemandaBaja + StockBajo

Resultados del período

- **Tiendas totales:** 80
- **Corte StockFinal:** 47.252
- **Corte Consumo:** 326.174

Conteo por cuadrante:

- DemandaAlta + StockAlto: 34
- DemandaBaja + StockBajo: 34
- DemandaBaja + StockAlto: 6
- DemandaAlta + StockBajo: 6

La mayoría de las tiendas queda en cuadrantes “esperables” (alta demanda con alto stock, o baja demanda con bajo stock).
Los cuadrantes minoritarios marcan posibles desbalances operativos:

- **DemandaAlta + StockBajo:** riesgo de quiebre por subabastecimiento.
- **DemandaBaja + StockAlto:** riesgo de sobrestock por exceso de inventario.

```python
# Calculamo la relación entre el inventario remanente y la demanda anual por punto de venta para identificar desequilibrios en la distribución regional de mercadería.
inv_store = (
    balance
    .groupby("Store", as_index=False)
    .agg(
        StockFinal=("StockFinal","sum"),
        Consumo=("Consumo","sum")
    )
)

# Cortes (mediana o p75/p25)
if CORTE_TIENDA == "mediana":
    corte_stock = inv_store["StockFinal"].median()
    corte_consumo = inv_store["Consumo"].median()
else:
    corte_stock = inv_store["StockFinal"].quantile(0.75)
    corte_consumo = inv_store["Consumo"].quantile(0.75)

inv_store["NivelStock"] = np.where(inv_store["StockFinal"] >= corte_stock, "StockAlto", "StockBajo")
inv_store["NivelDemanda"] = np.where(inv_store["Consumo"] >= corte_consumo, "DemandaAlta", "DemandaBaja")

inv_store["TipoTienda"] = inv_store["NivelDemanda"] + " + " + inv_store["NivelStock"]

print("TIPOS DE TIENDA (4 CUADRANTES)")
print("Tiendas totales:", len(inv_store))
print(f"Corte StockFinal: {corte_stock:,.0f} | Corte Consumo: {corte_consumo:,.0f}")
print("Conteo de tiendas por tipo:")
display(inv_store["TipoTienda"].value_counts().to_frame("Tiendas"))

print("Top 10:")
display(inv_store.head(10))
```
✓ 0.0s

```
TIPOS DE TIENDA (4 CUADRANTES)
Tiendas totales: 80
Corte StockFinal: 47,252 | Corte Consumo: 326,174
Conteo de tiendas por tipo:
```

TipoTienda	Tiendas
DemandaAlta + StockAlto	34
DemandaBaja + StockBajo	34
DemandaBaja + StockAlto	6
DemandaAlta + StockBajo	6

```
Top 10:
```

	Store	StockFinal	Consumo	NivelStock	NivelDemanda	TipoTienda
0	1	79827.0	558588.0	StockAlto	DemandaAlta	DemandaAlta + StockAlto
1	2	56671.0	417757.0	StockAlto	DemandaAlta	DemandaAlta + StockAlto
2	3	8470.0	35111.0	StockBajo	DemandaBaja	DemandaBaja + StockBajo
3	4	60570.0	269520.0	StockAlto	DemandaBaja	DemandaBaja + StockAlto
4	5	21708.0	124104.0	StockBajo	DemandaBaja	DemandaBaja + StockBajo
5	6	48303.0	461757.0	StockAlto	DemandaAlta	DemandaAlta + StockAlto
6	7	50780.0	409837.0	StockAlto	DemandaAlta	DemandaAlta + StockAlto

6- Composición del surtido por tipo de tienda (ABC)

Se analizó la profundidad y composición del catálogo activo en cada tipo de tienda, considerando únicamente los productos con consumo positivo.

Para cada cuadrante (Demanda/Stock) se calculó:

- Cantidad de marcas activas.
- Distribución porcentual por categoría ABC dentro de cada tipo de tienda.

Esto permite entender no sólo cuánto stock tiene cada tienda, sino **qué tipo de productos lo componen**.

DemandaAlta + StockAlto

- Alta concentración en categoría C (66,67%).
- Menor proporción en categoría A (14,17%).

DemandaAlta + StockBajo

- Distribución más equilibrada entre A, B y C.
- Mayor presencia relativa de categorías A y B respecto al grupo anterior.

DemandaBaja + StockAlto

- Predominio de categoría C (36,22%).
- Participación intermedia de B y menor de A.

DemandaBaja + StockBajo

- Fuerte concentración en categoría C (41,08%).

En general, los cuadrantes muestran una mayor presencia de productos categoría C, lo cual es consistente con la estructura global del catálogo.
Sin embargo:

- Las tiendas con **DemandaAlta + StockAlto** concentran gran volumen de productos C, lo que puede diluir el foco en productos estratégicos.
- Las tiendas con **DemandaAlta + StockBajo** presentan una composición más balanceada, lo que sugiere mayor alineación entre demanda y criticidad económica.

Este análisis permite evaluar si el surtido acompaña correctamente la dinámica de cada tipo de tienda o si existen oportunidades de ajuste en la mezcla de categorías.

```python
store_brand = (
    balance
    .groupby(["Store","Brand"], as_index=False)
    .agg(
        StockFinal=("StockFinal","sum"),
        Consumo=("Consumo","sum")
    )
)

# 2) Pegamos ABC
store_brand = store_brand.merge(abc[["Brand","CategoriaABC"]], on="Brand", how="left")
store_brand["CategoriaABC"] = store_brand["CategoriaABC"].fillna("SinABC")

# 3) Pegamos tipo de tienda
store_brand = store_brand.merge(inv_store[["Store","TipoTienda"]], on="Store", how="left")

print("ABC POR TIPO DE TIENDA")

# Conteo de brands distintos por tipo y ABC
mix_tienda = (
    store_brand[store_brand["Consumo"] > 0]  # solo productos que realmente se mueven en esa tienda
    .groupby(["TipoTienda","CategoriaABC"])["Brand"]
    .nunique()
    .reset_index(name="Brands_Unicos")
)

# % dentro de cada tipo de tienda
mix_tienda["%_en_TipoTienda"] = (
    mix_tienda.groupby("TipoTienda")["Brands_Unicos"]
    .apply(lambda s: (s / s.sum() * 100).round(2))
    .reset_index(drop=True)
)

display(mix_tienda.sort_values(["TipoTienda","CategoriaABC"]))
```

	TipoTienda	CategoriaABC	Brands_Unicos	%_en_TipoTienda
0	DemandaAlta + StockAlto	A	1550	14.17
1	DemandaAlta + StockAlto	B	2096	19.16
2	DemandaAlta + StockAlto	C	7294	66.67
3	DemandaAlta + StockBajo	A	1540	31.51
4	DemandaAlta + StockBajo	B	1749	35.78
5	DemandaAlta + StockBajo	C	1599	32.71
6	DemandaBaja + StockAlto	A	1540	28.10
7	DemandaBaja + StockAlto	B	1955	35.68
8	DemandaBaja + StockAlto	C	1985	36.22
9	DemandaBaja + StockBajo	A	1540	26.23
10	DemandaBaja + StockBajo	B	1919	32.69
11	DemandaBaja + StockBajo	C	2412	41.08

7- Relación entre consumo y stock por tienda

Se construyó un gráfico de dispersión para visualizar la relación entre el **consumo total** y el **stock final** a nivel tienda.

- Eje X: Consumo total por tienda
- Eje Y: Stock final total por tienda
- Cada punto representa una tienda
- El color identifica el cuadrante (Tipo Tienda)

Se incorporaron líneas de corte (según el criterio definido previamente) que dividen el gráfico en cuatro cuadrantes:

- DemandaAlta + StockAlto
- DemandaAlta + StockBajo
- DemandaBaja + StockAlto
- DemandaBaja + StockBajo

Observaciones principales

- La mayor concentración de tiendas se encuentra en los cuadrantes equilibrados (alta demanda con alto stock, y baja demanda con bajo stock).
- Un grupo reducido aparece en **DemandaAlta + StockBajo**, lo que sugiere posible riesgo de subabastecimiento.
- También hay casos en **DemandaBaja + StockAlto**, lo que podría indicar acumulación innecesaria de inventario.

El gráfico confirma que, en términos generales, la asignación de inventario acompaña el nivel de demanda.

Los desbalances identificados son puntuales y se concentran en un número reducido de tiendas, lo que facilita una intervención focalizada.

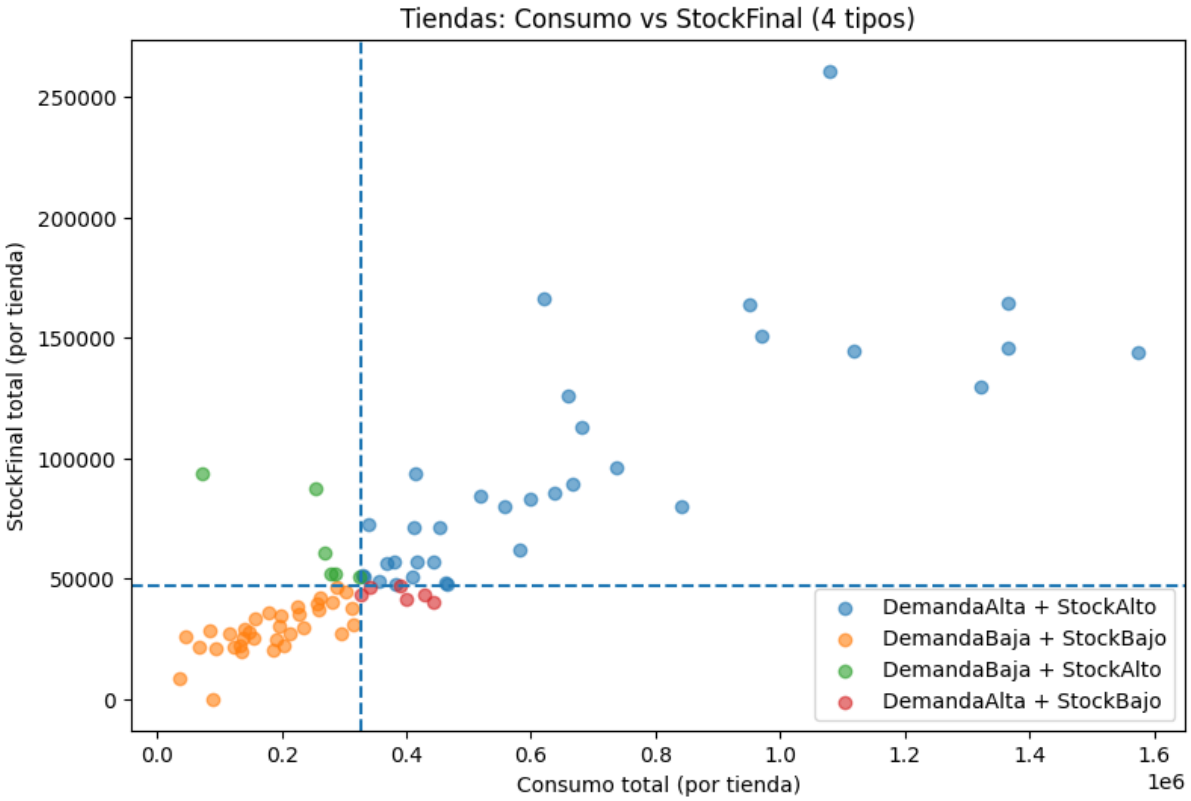
Esta visualización complementa el análisis numérico y permite identificar rápidamente patrones operativos y posibles desajustes regionales.

```
# Filtramos los productos del cuartil superior de demanda con inventario agotado para identificar las brechas de stock que generan la mayor pérdida de ingresos potencial.

plt.figure(figsize=(9,6))

tipos = inv_store["TipoTienda"].unique()
for t in tipos:
    sub = inv_store[inv_store["TipoTienda"] == t]
    plt.scatter(sub["Consumo"], sub["StockFinal"], alpha=0.6, label=t)

plt.axvline(corte_consumo, linestyle="--")
plt.axhline(corte_stock, linestyle="--")
plt.title("Tiendas: Consumo vs StockFinal (4 tipos)")
plt.xlabel("Consumo total (por tienda)")
plt.ylabel("StockFinal total (por tienda)")
plt.legend()
plt.show()
```



8- Cobertura de inventario por tipo de tienda y categoría ABC

Se calculó la relación entre el stock disponible y la demanda anual para cada combinación de **TipoTienda** y **Categoría ABC**.

La métrica utilizada fue:

Cobertura (%) = (StockFinal / Consumo) × 100

Este indicador permite estimar cuánto inventario existe en relación con el nivel de demanda.

Lectura del indicador

- 100% → stock equivalente a un año de consumo.
- 50% → medio año de cobertura.
- Valores altos → posible acumulación.
- Valores bajos → riesgo de quiebre.

Resultados relevantes

- Las categorías **C** presentan las coberturas más altas en casi todos los tipos de tienda.
- Las categorías **A** muestran coberturas más ajustadas, especialmente en tiendas con alta demanda.
- En el cuadrante **DemandaBaja + StockAlto**, la cobertura en categoría C alcanza los valores más elevados.

El inventario estratégico (categoría A) mantiene niveles de cobertura relativamente controlados, lo que es consistente con una gestión más activa.

En cambio, los productos de categoría C tienden a acumular mayor stock relativo, especialmente en tiendas de baja demanda, lo que puede indicar menor dinamismo operativo.

Este análisis permite evaluar no solo cuánto stock hay, sino si ese stock está alineado con la relevancia económica de los productos y el perfil de cada tienda.

```
mix_tienda = (
    store_brand
    .groupby(["TipoTienda", "CategoriaABC"], as_index=False)
    .agg(
        StockFinal=("StockFinal","sum"),
        Consumo=("Consumo","sum")
    )
)

# Ratio expresado en %
mix_tienda["Cobertura_%"] = np.where(
    mix_tienda["Consumo"] > 0,
    (mix_tienda["StockFinal"] / mix_tienda["Consumo"]) * 100,
    np.nan
).round(2)

display(mix_tienda.sort_values(["TipoTienda","CategoriaABC"]))
```

	TipoTienda	CategoriaABC	StockFinal	Consumo	Cobertura_%
0	DemandaAlta + StockAlto	A	1823089.0	16311728.0	11.18
1	DemandaAlta + StockAlto	B	855614.0	4919958.0	17.39
2	DemandaAlta + StockAlto	C	568615.0	1571853.0	36.17
3	DemandaAlta + StockBajo	A	172485.0	1871143.0	9.22
4	DemandaAlta + StockBajo	B	62926.0	381037.0	16.51
5	DemandaAlta + StockBajo	C	26276.0	81817.0	32.12
6	DemandaBaja + StockAlto	A	254214.0	1141505.0	22.27
7	DemandaBaja + StockAlto	B	103398.0	278938.0	37.07
8	DemandaBaja + StockAlto	C	38662.0	65589.0	58.95
9	DemandaBaja + StockBajo	A	717803.0	5195769.0	13.82
10	DemandaBaja + StockBajo	B	196008.0	890615.0	22.01
11	DemandaBaja + StockBajo	C	66686.0	207924.0	32.07

Conclusión del bloque

El análisis permitió evaluar el estado operativo del inventario desde tres dimensiones: disponibilidad, riesgo y asignación por tienda.

En términos generales:

- No se identifican problemas estructurales de sobrestock.
- El quiebre operativo existe, pero se concentra principalmente en productos de baja criticidad (categoría C).
- El riesgo crítico (alto consumo sin stock) es reducido y no representa una amenaza significativa sobre la facturación total.
- La mayoría de las tiendas presenta coherencia entre nivel de demanda y nivel de inventario.
- Las coberturas más elevadas se observan en productos de baja rotación, especialmente en tiendas de baja demanda.

En conjunto, el inventario muestra un estado estable, con desbalances puntuales más que sistémicos.
Las oportunidades de mejora se concentran en ajustes finos: priorización de productos estratégicos y optimización del surtido en categorías de baja rotación.

Este bloque complementa el análisis de rotación, aportando una visión operativa del riesgo actual y del uso del capital invertido en stock.

BLOQUE 6 – Análisis de Lead Time

Objetivo: El objetivo de este bloque es analizar el proceso de abastecimiento desde una perspectiva logística, evaluando el desempeño de los proveedores en términos de tiempos de entrega y su impacto en la operación.

Se busca:

- Medir el tiempo promedio que tardan los proveedores en entregar pedidos.
- Identificar la variabilidad en los plazos de entrega.
- Detectar posibles riesgos logísticos asociados a demoras.
- Evaluar si la estructura de abastecimiento presenta señales de ineficiencia o vulnerabilidad.

Este análisis permite comprender si el sistema de compras opera de manera estable o si existen factores que podrían afectar la disponibilidad de stock en las tiendas.

1 – Identificación y medición del universo de proveedores

Se trabajó sobre el dataset `Compra Final`, calculando el Lead Time como la diferencia en días entre la fecha de pedido (`PODate`) y la fecha de recepción (`ReceivingDate`).

A partir de esta métrica:

- Se identificaron **126 proveedores únicos** que realizaron entregas durante el período analizado.
- Se calculó el Lead Time promedio por proveedor.
- Se obtuvieron estadísticas descriptivas para comprender el comportamiento general de los tiempos de entrega.

Resultados principales:

- **Lead Time promedio general:** aproximadamente 7,75 días.
- **Lead Time mínimo observado:** 5 días.
- **Lead Time máximo observado:** 13 días.
- La dispersión (desvío estándar bajo) indica que la mayoría de los proveedores entregan en un rango relativamente estable.

Este primer análisis permitió establecer una base cuantitativa del desempeño logístico antes de avanzar hacia clasificaciones y cruces con otras variables.

```python
# Promedio de lead time por proveedor

Compra_final["LeadTimeDays"] = (Compra_final["ReceivingDate"] - Compra_final["PODate"]).dt.days

# Promedio de lead time por proveedor
vendor_lt = (
    Compra_final
    .groupby("VendorNumber")["LeadTimeDays"]
    .mean()
    .reset_index(name="LT_Promedio")
)

print("Cantidad de proveedores analizados:", vendor_lt["VendorNumber"].nunique())

print("\nEstadísticas Lead Time (días):")
print(vendor_lt["LT_Promedio"].describe(percentiles=[0.25, 0.50, 0.75, 0.90]).to_string())

print("\nLead Time mínimo proveedor:", vendor_lt["LT_Promedio"].min())
print("Lead Time máximo proveedor:", vendor_lt["LT_Promedio"].max())
```
✓ 0.2s

```
Cantidad de proveedores analizados: 126

Estadísticas Lead Time (días):
count    126.000000
mean       7.748599
std        0.780303
min        5.000000
25%        7.470760
50%        7.666354
75%        7.949221
90%        8.278595
max       13.000000

Lead Time mínimo proveedor: 5.0
Lead Time máximo proveedor: 13.0
```

2 – Clasificación de proveedores según velocidad de entrega

Una vez calculado el Lead Time promedio por proveedor, se procedió a clasificar a los proveedores según su velocidad de entrega.

Se definieron tres categorías:

- **Rápido:** proveedores con Lead Time promedio menor o igual a 6 días.
- **Moderado:** proveedores con Lead Time promedio mayor a 6 días y hasta 9 días.
- **Lento:** proveedores con Lead Time promedio superior a 9 días.

Resultados de la clasificación:

- **118 proveedores** se clasifican como *Moderado*.
- **5 proveedores** se clasifican como *Lento*.
- **3 proveedores** se clasifican como *Rápido*.

La estructura logística se encuentra fuertemente concentrada en proveedores de velocidad moderada, representando aproximadamente el 94% del total analizado. Los proveedores rápidos y lentos constituyen una proporción marginal dentro del sistema.

Esto sugiere que el abastecimiento opera bajo un esquema relativamente homogéneo en términos de tiempos de entrega, sin una alta dependencia de proveedores extremadamente rápidos ni una exposición significativa a proveedores con demoras prolongadas.

```python
# Categorización de proveedores por velocidad de entrega

vendor_lt["CategoriaEntregaProveedor"] = np.where(
    vendor_lt["LT_Promedio"] <= 6, "Rapido",
    np.where(
        vendor_lt["LT_Promedio"] <= 9, "Moderado",
        "Lento"
    )
)

print("\nConteo por categoría:")
print(vendor_lt["CategoriaEntregaProveedor"].value_counts())
```

✓ 0.0s

```
Conteo por categoría:
CategoriaEntregaProveedor
Moderado    118
Lento         5
Rapido        3
Name: count, dtype: int64
```

3 – Distribución detallada del Lead Time (por días)

Además de la clasificación general (Rápido / Moderado / Lento), se realizó un análisis más granular del Lead Time promedio por proveedor.

Para ello:

- Se redondeó el Lead Time promedio a días enteros.
- Se limitó el rango entre 5 y 13 días para evitar valores extremos.
- Se analizó la distribución de proveedores según la cantidad exacta de días promedio de entrega.

Resultados:

- 2 proveedores entregan en promedio en **5 días**.
- 1 proveedor entrega en **6 días**.
- 33 proveedores entregan en **7 días**.
- 83 proveedores entregan en **8 días**.
- 3 proveedores entregan en **9 días**.

- 3 proveedores entregan en **10 días**.
- 1 proveedor entrega en **13 días**.

Interpretación:

La mayoría de los proveedores se concentra entre **7 y 8 días**, siendo 8 días el valor más frecuente.

Esto confirma que el sistema de abastecimiento presenta una estructura relativamente estable y poco dispersa, con una fuerte concentración en un rango acotado de tiempos de entrega.

No se observa una presencia significativa de proveedores extremadamente lentos ni una dispersión amplia en los plazos logísticos.

```python
# Redondeamos el promedio para trabajar por días enteros

vendor_lt["LT_Round"] = vendor_lt["LT_Promedio"].round().astype(int)

# Limitar rango: <=5 → 5 | >=13 → 13
vendor_lt["LT_Categoria"] = vendor_lt["LT_Round"].clip(lower=5, upper=13)

print("\nConteo por categoría LT:")
print(vendor_lt["LT_Categoria"].value_counts().sort_index())
```
✓ 0.0s

```
Conteo por categoría LT:
LT_Categoria
5      2
6      1
7     33
8     83
9      3
10     3
13     1
Name: count, dtype: int64
```

4 – Análisis de exclusividad de productos por proveedor

Con el objetivo de comprender la estructura de dependencia entre productos y proveedores, se analizó cuántos proveedores abastecen cada marca (`Brand`).

Para cada producto se contabilizó la cantidad de proveedores distintos que lo suministran y se clasificó en:

- **Exclusiva:** producto abastecido por un único proveedor.
- **Compartida:** producto abastecido por más de un proveedor.
- **10.635 productos** son abastecidos por un único proveedor (Exclusiva).
- **29 productos** son abastecidos por más de un proveedor (Compartida).

La estructura de abastecimiento presenta un nivel muy alto de exclusividad. La gran mayoría de los productos dependen de un solo proveedor, lo que implica:

- Baja redundancia en la cadena de suministro.
- Alta dependencia individual por producto.
- Riesgo potencial ante fallas o demoras de un proveedor específico.

La existencia de solo 29 productos compartidos indica que la empresa tiene muy poca diversificación de fuentes para un mismo producto.

```python
brand_vendor_count = (
    Compra_final
    .groupby("Brand")["VendorNumber"]
    .nunique()
    .reset_index(name="Cantidad_Proveedores")
)

brand_vendor_count["Tipo"] = (
    brand_vendor_count["Cantidad_Proveedores"]
    .apply(lambda x: "Exclusiva" if x == 1 else "Compartida")
)

brand_vendor_count["Tipo"].value_counts()
```

✓ 0.1s

```
Tipo
Exclusiva     10635
Compartida       29
Name: count, dtype: int64
```

5 – Análisis de proveedores con marcas compartidas

Luego de identificar que únicamente **29 productos** son abastecidos por más de un proveedor, se analizó cuántos proveedores participan en este esquema de abastecimiento compartido.

Resultado:

- **13 proveedores** trabajan con al menos una marca compartida.

Esto implica que, dentro de los 126 proveedores totales, solo un grupo reducido participa en productos con abastecimiento alternativo.

Distribución de marcas compartidas por proveedor y categoría ABC

Se profundizó el análisis identificando, para estos 13 proveedores, cuántas marcas compartidas gestionan y a qué categoría ABC pertenecen (A, B o C).

El objetivo fue comprender si los productos estratégicos (categoría A) presentan mayor diversificación de proveedores o si la compartición se concentra en categorías de menor impacto.

Observaciones generales:

- La participación en marcas compartidas es limitada y distribuida en pocos proveedores.
- Algunos proveedores gestionan marcas compartidas en distintas categorías ABC.
- No se observa una concentración dominante de marcas compartidas exclusivamente en la categoría A.

Interpretación:

El sistema presenta muy baja redundancia en productos estratégicos.
La mayoría de los productos —incluyendo muchos de categoría A— dependen de un único proveedor.

Esto refuerza la conclusión de que la estructura de abastecimiento es mayormente exclusiva y con escasa diversificación de fuentes.

```python
# 1) Marcas compartidas (las 29)
marcas_comp = brand_vendor_count.loc[
    brand_vendor_count["Cantidad_Proveedores"] > 1, "Brand"
]

# 2) Resumen: por proveedor y categoría ABC, cuántas marcas distintas tiene
resumen_vendor_abc = (
    Compra_final.loc[
        Compra_final["Brand"].isin(marcas_comp),
        ["VendorNumber","VendorName","Brand"]
    ]
    .drop_duplicates()
    .merge(
        abc[["Brand","CategoriaABC"]].drop_duplicates(),
        on="Brand",
        how="left"
    )
    .groupby(["VendorNumber","VendorName","CategoriaABC"])["Brand"]
    .nunique()
    .reset_index(name="Cantidad_Marcas")
    .sort_values(["VendorName","VendorNumber","CategoriaABC"])
)

print(resumen_vendor_abc.to_string(index=False))
```
✓ 0.1s

VendorNumber	VendorName	CategoriaABC	Cantidad_Marcas
1128	BROWN-FORMAN CORP	A	7
1128	BROWN-FORMAN CORP	B	1
1128	BROWN-FORMAN CORP	C	3
201359	FLAVOR ESSENCE INC	C	1
9744	FREDERICK WILDMAN & SONS	A	1
9744	FREDERICK WILDMAN & SONS	B	1
9744	FREDERICK WILDMAN & SONS	C	2
2	IRA GOLDMAN AND WILLIAMS, LLP	C	1
4692	KOBRAND CORPORATION	B	1
9552	M S WALKER INC	B	1
9552	M S WALKER INC	C	4
4425	MARTIGNETTI COMPANIES	B	2
4425	MARTIGNETTI COMPANIES	C	1
4425	MARTIGNETTI COMPANIES	B	4
4425	MARTIGNETTI COMPANIES	C	1
6830	PARK STREET IMPORTS LLC	B	1
10754	PERFECTA WINES	A	1
10754	PERFECTA WINES	B	3
10754	PERFECTA WINES	C	2
7153	PINE STATE TRADING CO	B	1
7153	PINE STATE TRADING CO	C	3
8004	SAZERAC CO INC	A	7
8004	SAZERAC CO INC	B	2
8004	SAZERAC CO INC	C	3
3089	SIDNEY FRANK IMPORTING CO	B	1
9165	ULTRA BEVERAGE COMPANY LLP	B	2
9165	ULTRA BEVERAGE COMPANY LLP	C	1

6 – Relación entre velocidad del proveedor y categoría ABC del producto

Con el objetivo de analizar si los productos estratégicos (categoría A) dependen de proveedores más rápidos o más lentos, se realizó un cruce entre:

- **Categoría de Lead Time del proveedor** (expresada en días promedio: 5, 6, 7, 8, 9, 10, 13).
- **Categoría ABC del producto** (A, B o C).

Para evitar duplicaciones por compras repetidas, se trabajó sobre la relación única proveedor–producto.
Luego, se calculó el porcentaje que representa cada categoría de Lead Time dentro del total de cada grupo ABC (es decir, A = 100%, B = 100%, C = 100%).

- La categoría **8 días** concentra la mayor proporción de productos en todas las categorías ABC:
    - 81,58% de los productos A.
    - 76,79% de los productos B.
    - 71,22% de los productos C.

- La categoría **7 días** representa:
    - 18,42% de los productos A.
    - 23,21% de los productos B.
    - 28,46% de los productos C.
- Las categorías 5, 6, 9, 10 y 13 días tienen una participación marginal.

El abastecimiento de productos estratégicos (categoría A) no depende de proveedores significativamente más rápidos que el resto.

Por el contrario:

- La gran mayoría de los productos A, B y C están abastecidos por proveedores con un Lead Time promedio de 8 días.
- No se observa una asignación diferenciada de velocidad logística para productos de mayor importancia.

Esto refuerza la conclusión previa:

El sistema de abastecimiento es homogéneo en términos de tiempos de entrega, independientemente de la criticidad del producto.

No se identifica una política logística diferenciada para productos estratégicos.

```
# Mapas (proveedor -> LT, producto -> ABC)
lt_map  = vendor_lt[["VendorNumber","LT_Categoria"]].drop_duplicates()
abc_map = abc[["Brand","CategoriaABC"]].drop_duplicates()

# Base: relación proveedor-marca (evitamos duplicar por compras repetidas)
base = (
    Compra_final[["VendorNumber","Brand"]]
    .drop_duplicates()
    .merge(lt_map, on="VendorNumber", how="left")
    .merge(abc_map, on="Brand", how="left")
)

# Conteo por (LT proveedor, ABC producto)
t = (
    base.groupby(["LT_Categoria","CategoriaABC"])
    .size()
    .reset_index(name="Cantidad")
)

# % dentro de cada ABC (A=100%, B=100%, C=100%)
t["Pct_del_total_ABC"] = (
    t["Cantidad"] / t.groupby("CategoriaABC")["Cantidad"].transform("sum") * 100
).round(2)

# Orden como querés: 1) LT proveedor, 2) ABC producto, 3) porcentaje
t = t.sort_values(["LT_Categoria","CategoriaABC"])

print(t.to_string(index=False))
```

LT_Categoria	CategoriaABC	Cantidad	Pct_del_total_ABC
5	C	12	0.17
6	C	1	0.01
7	A	287	18.42
7	B	489	23.21
7	C	2000	28.46
8	A	1271	81.58
8	B	1618	76.79
8	C	5005	71.22
9	C	5	0.07
10	C	4	0.06

7 – Volumen anual de compra por proveedor

Con el objetivo de identificar el peso económico de cada proveedor dentro del sistema de abastecimiento, se calculó el volumen total de compras anual por proveedor, utilizando el monto total facturado (`Dollars`).

Se agruparon las compras por proveedor y se ordenaron de mayor a menor para identificar los actores más relevantes en términos financieros.

Principales proveedores por volumen anual

Entre los proveedores con mayor volumen de compra se destacan:

- **DIAGEO NORTH AMERICA INC**
- **MARTIGNETTI COMPANIES**
- **JIM BEAM BRANDS COMPANY**
- **PERNOD RICARD USA**
- **BACARDI USA INC**

Estos proveedores concentran una proporción significativa del gasto anual en compras.

El análisis muestra que el sistema de abastecimiento presenta una concentración económica en un grupo reducido de proveedores.

Esto implica:

- Alta dependencia financiera de ciertos proveedores estratégicos.
- Mayor impacto operativo ante posibles demoras o interrupciones de estos actores.
- Necesidad de monitorear no solo la velocidad de entrega, sino también la exposición económica.

Este paso complementa el análisis de Lead Time, ya que permite evaluar no solo cuánto tardan los proveedores, sino cuánto representan en términos de volumen de compra.

```python
# Volumen de comprar por proveedor.

volumen_proveedor = (
    Compra_final
    .groupby(["VendorNumber","VendorName"])["Dollars"]
    .sum()
    .reset_index()
    .sort_values("Dollars", ascending=False)
)

volumen_proveedor.head(10)
```

✓ 0.4s

	VendorNumber	VendorName	Dollars
42	3960	DIAGEO NORTH AMERICA INC	50959796.85
44	4425	MARTIGNETTI COMPANIES	27821473.91
98	12546	JIM BEAM BRANDS COMPANY	24203151.05
102	17035	PERNOD RICARD USA	24124091.56
7	480	BACARDI USA INC	17624378.72
16	1392	CONSTELLATION BRANDS INC	15573917.90
12	1128	BROWN-FORMAN CORP	13529433.08
83	9165	ULTRA BEVERAGE COMPANY LLP	13210613.93
36	3252	E & J GALLO WINERY	12289608.09
86	9552	M S WALKER INC	10935817.30

8 – Volumen anual de compra por categoría de proveedor

Luego de clasificar a los proveedores según su velocidad de entrega (Rápido, Moderado, Lento), se analizó el volumen anual de compra asociado a cada categoría.

Se cruzó el monto total comprado (`Dollars`) con la categoría logística del proveedor y se sumó el total anual por grupo.

- **Moderado:** 32.187.348,82 USD
- **Lento:** 20.358,95 USD
- **Rápido:** 8.057,76 USD

El análisis muestra una concentración casi absoluta del volumen de compra en proveedores de categoría **Moderado**.

En términos prácticos:

- Más del 99% del gasto anual se realiza con proveedores cuyo Lead Time promedio se ubica entre 6 y 9 días.
- La participación económica de proveedores rápidos y lentos es marginal.

Esto refuerza una conclusión estructural:

El sistema de abastecimiento no depende significativamente de proveedores extremadamente rápidos ni presenta una exposición relevante a proveedores lentos.

La operación logística está claramente centralizada en un rango de entrega moderado y relativamente estable.

```python
# Volumen de compra por categoria de proveerdor.

volumen_categoria = (
    Compra_final
    .merge(
        vendor_lt[["VendorNumber","CategoriaEntregaProveedor"]],
        on="VendorNumber",
        how="left"
    )
    .groupby("CategoriaEntregaProveedor")["Dollars"]
    .sum()
    .reset_index()
    .sort_values("Dollars", ascending=False)
)

print(volumen_categoria.to_string(index=False))
```

✓ 5.9s

```
CategoriaEntregaProveedor      Dollars
                 Moderado 321872348.82
                    Lento     20358.95
                   Rapido      8057.76
```

9 – Frecuencia de compra por producto según categoría ABC

Con el objetivo de analizar la dinámica de reposición, se calculó la frecuencia de órdenes de compra por producto (cantidad de veces que cada marca fue pedida durante el año).

Luego, esta frecuencia se agrupó por categoría ABC para identificar patrones de comportamiento en productos estratégicos (A), intermedios (B) y de baja rotación (C).

Categoría	Frecuencia media	Mediana	Máximo	Cantidad de productos
A	51,25	54	55	1.550
B	40,27	47	55	2.097
C	9,28	4	55	7.017

- Los productos **A** presentan una frecuencia de compra muy alta (en promedio más de 51 órdenes anuales).
- Los productos **B** también muestran una frecuencia elevada, aunque menor que A.
- Los productos **C** tienen una frecuencia significativamente inferior, con una mediana de apenas 4 órdenes por año.

Esto indica que:

- Los productos estratégicos (A) se reponen de manera constante y sostenida.
- Los productos C presentan reposiciones esporádicas y menor presión operativa.
- El sistema de compras responde coherentemente a la importancia relativa del producto.

Este análisis permite detectar posibles compras fragmentadas o comportamientos reactivos, especialmente si se combinará luego con el análisis Compras y Consumo.

```
# Resumen por categoría ABC.

frecuencia_producto = frecuencia_producto.merge(
    abc[["Brand","CategoriaABC"]],
    on="Brand",
    how="left"
)

resumen_frecuencia_abc = (
    frecuencia_producto
    .groupby("CategoriaABC")["Frecuencia_Ordenes"]
    .agg(["mean","median","max","count"])
    .reset_index()
)

print(resumen_frecuencia_abc)
```

✓ 0.1s

```
  CategoriaABC       mean  median  max  count
0            A  51.250968    54.0   55   1550
1            B  40.274678    47.0   55   2097
2            C   9.277469     4.0   55   7017
```

10 – Cobertura de compra por categoría ABC

Con el objetivo de evaluar si la política de compras acompaña la importancia estratégica de los productos, se analizó qué proporción de productos de cada categoría ABC fue efectivamente comprada durante el año.

Para cada categoría se identificó:

- Total de productos existentes.
- Cantidad de productos que registraron compras.
- Cantidad de productos que no fueron comprados.
- Porcentaje de cobertura dentro de la categoría.

Categoría	Total productos	Productos comprados	No comprados	% Comprados
A	1.550	1.550	0	100%
B	2.097	2.097	0	100%
C	7.856	7.017	839	89,32%

- La empresa compra el **100% de los productos estratégicos (A)**.
- También mantiene cobertura total en la categoría B.
- En la categoría C, aproximadamente el **10,68% de los productos no fue comprado** durante el año.

Esto indica:

- Fuerte alineación entre política de compras y criticidad del producto.
- No se observan abandonos ni desatención en productos estratégicos.
- La no reposición se concentra en productos de baja importancia (C), lo cual es coherente desde una perspectiva operativa.

Este resultado sugiere que el sistema prioriza correctamente la reposición de productos de mayor impacto.

```
# Total productos por categoría ABC

total_abc = (
    abc
    .groupby("CategoriaABC")["Brand"]
    .nunique()
    .reset_index(name="Total_Productos_ABC")
)
```

```python
# Productos comprados (marcas únicas compradas)

marcas_compradas = Compra_final["Brand"].drop_duplicates()

comprados_abc = (
    abc[abc["Brand"].isin(marcas_compradas)]
    .groupby("CategoriaABC")["Brand"]
    .nunique()
    .reset_index(name="Productos_Comprados")
)

# Unir y calcular porcentaje

tabla_abc = total_abc.merge(
    comprados_abc,
    on="CategoriaABC",
    how="left"
)

tabla_abc["Productos_No_Comprados"] = (
    tabla_abc["Total_Productos_ABC"] -
    tabla_abc["Productos_Comprados"]
)

tabla_abc["Pct_Comprados_dentro_ABC"] = (
    tabla_abc["Productos_Comprados"] /
    tabla_abc["Total_Productos_ABC"] * 100
).round(2)

print(" Porcentaje de productos comprados dentro de cada categoría ABC")
print(tabla_abc.to_string(index=False))
```

11 – Volumen total comprado por categoría ABC

Además de analizar qué productos fueron comprados, se evaluó el volumen total de unidades adquiridas durante el año por cada categoría ABC.

Se calcularon:

- Cantidad de marcas distintas compradas por categoría.
- Cantidad total de unidades compradas.
- Porcentaje que representa cada categoría sobre el total anual comprado.

Categoría	Total Marcas	Unidades Compradas	% sobre total anual
A	1.550	24.889.979	74,11%
B	2.097	6.616.899	19,70%
C	7.017	2.077.499	6,19%

- La categoría **A** concentra aproximadamente el **74% del volumen total comprado**.
- La categoría **B** representa cerca del 20%.
- La categoría **C**, a pesar de tener la mayor cantidad de productos, solo representa el 6% del volumen.

Esto confirma que la estructura de compras está alineada con la lógica ABC:

- Pocos productos (A) concentran la mayor parte del volumen.
- Muchos productos (C) tienen bajo peso operativo.
- La política de abastecimiento prioriza correctamente los productos estratégicos.

Desde una perspectiva de gestión, el sistema muestra coherencia entre importancia del producto y asignación de recursos de compra.

```python
# Cantidad total comprada por categoría ABC

compras_abc = (
    Compra_final
    .merge(
        abc[["Brand","CategoriaABC"]],
        on="Brand",
        how="left"
    )
    .groupby("CategoriaABC")
    .agg(
        Total_Marcas=("Brand","nunique"),
        Cantidad_Total_Comprada=("Quantity","sum")
    )
    .reset_index()
)

# Calcular total anual comprado (unidades)
total_anual = compras_abc["Cantidad_Total_Comprada"].sum()

# Calcular porcentaje sobre el total anual
compras_abc["%_sobre_Total_Anual"] = (
    compras_abc["Cantidad_Total_Comprada"] /
    total_anual * 100
).round(2)

print(" Cantidad total comprada por categoría ABC ")
print(compras_abc.sort_values("Cantidad_Total_Comprada", ascending=False).to_string(index=False))
```

✓ 1.4s

```
 Cantidad total comprada por categoría ABC
CategoriaABC  Total_Marcas  Cantidad_Total_Comprada  %_sobre_Total_Anual
           A          1550                 24889979                74.11
           B          2097                  6616899                19.70
           C          7017                  2077499                 6.19
```

12- Relación entre tipo de tienda y velocidad del proveedor

Con el objetivo de identificar si el nivel de stock de una tienda está condicionado por la velocidad del proveedor, se cruzaron dos segmentaciones previamente construidas:

1. **Tipo de tienda** (según nivel de demanda y nivel de stock):
    - DemandaAlta + StockAlto
    - DemandaAlta + StockBajo
    - DemandaBaja + StockAlto
    - DemandaBaja + StockBajo

2. **Categoría de proveedor** según velocidad de entrega:

    - Rápido
    - Moderado
    - Lento

Se calculó la proporción de órdenes que cada tipo de tienda realiza a proveedores de cada categoría logística.

En todos los tipos de tienda se observa una concentración prácticamente total en proveedores **Moderado**, con valores cercanos al 99%.

La participación de proveedores:

- **Lento** es marginal.
- **Rápido** es prácticamente inexistente en términos de volumen de órdenes.

No se identifica una relación directa entre bajo nivel de stock y dependencia de proveedores lentos.

Dado que todas las tiendas compran mayoritariamente a proveedores de categoría Moderado, el nivel de stock no parece estar condicionado por diferencias significativas en la velocidad logística.

Esto sugiere que:

- El sistema de abastecimiento es homogéneo.
- La variación en stock entre tiendas responde más a decisiones operativas o dinámicas de demanda que a diferencias en tiempos de entrega.

```
# Cruzamos tienda con categoría de proveedor
tienda_proveedor = (
    Compra_final
    .merge(
        vendor_lt[["VendorNumber","CategoriaEntregaProveedor"]],
        on="VendorNumber",
        how="left"
    )
    .groupby(["Store","CategoriaEntregaProveedor"])
    .size()
    .reset_index(name="Cantidad_Ordenes")
)

# Unimos con TipoTienda ya creado en inv_store
cruce_final = tienda_proveedor.merge(
    inv_store[["Store","TipoTienda"]],
    on="Store",
    how="left"
)
```

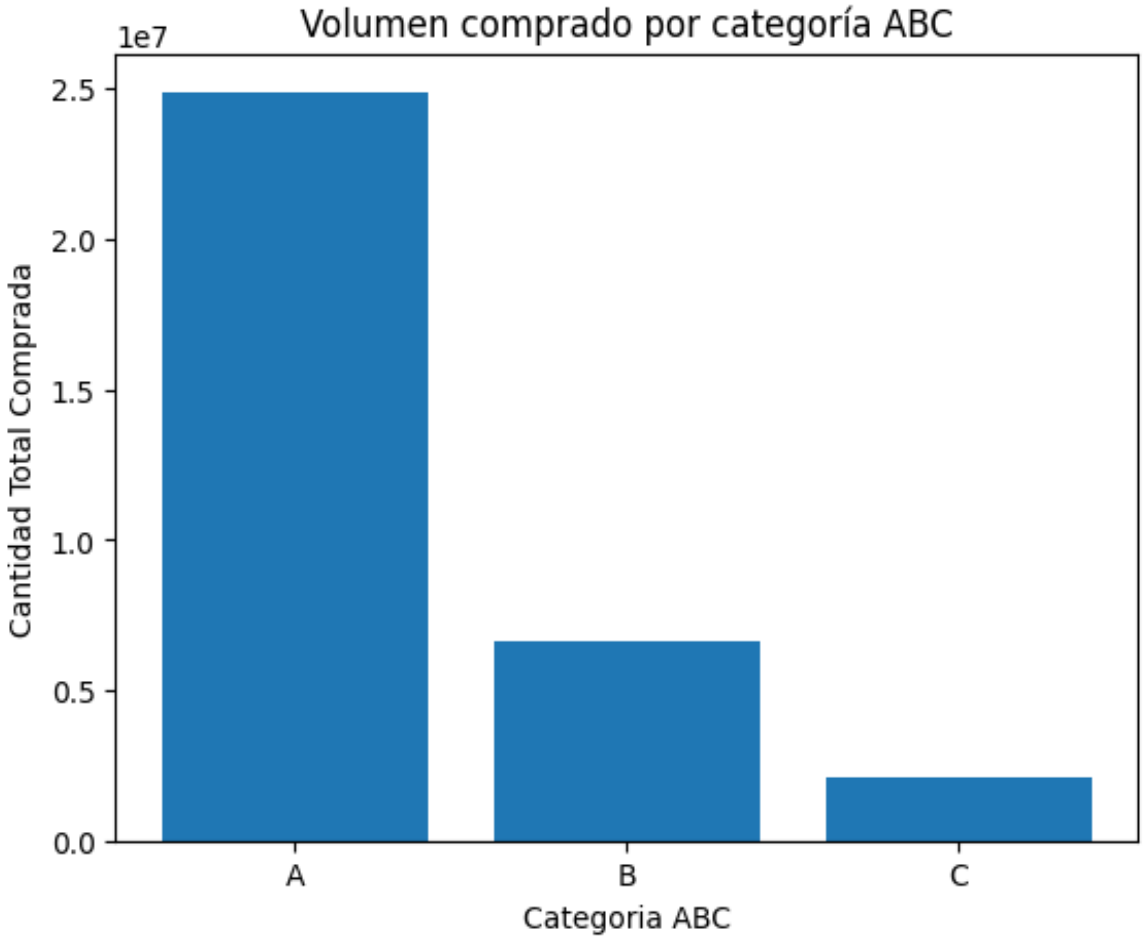
```python
# Distribución porcentual dentro de cada tipo de tienda
tabla_final = (
    cruce_final
    .groupby(["TipoTienda","CategoriaEntregaProveedor"])["Cantidad_Ordenes"]
    .sum()
    .reset_index()
)

tabla_final["Pct_dentro_TipoTienda"] = (
    tabla_final["Cantidad_Ordenes"] /
    tabla_final.groupby("TipoTienda")["Cantidad_Ordenes"].transform("sum")
) * 100

print("Distribución de tipo de proveedor dentro de cada tipo de tienda")
print(tabla_final.sort_values(["TipoTienda","CategoriaEntregaProveedor"]).to_string(index=False))
```

✓ 3.9s

```
Distribución de tipo de proveedor dentro de cada tipo de tienda
             TipoTienda CategoriaEntregaProveedor  Cantidad_Ordenes  Pct_dentro_TipoTienda
DemandaAlta + StockAlto                     Lento                45               0.002801
DemandaAlta + StockAlto                  Moderado           1606301              99.994148
DemandaAlta + StockAlto                    Rapido                49               0.003050
DemandaAlta + StockBajo                     Lento                 3               0.001685
DemandaAlta + StockBajo                  Moderado            178054              99.998315
DemandaBaja + StockAlto                     Lento                 6               0.005630
DemandaBaja + StockAlto                  Moderado            106560              99.994370
DemandaBaja + StockBajo                     Lento                 6               0.001246
DemandaBaja + StockBajo                  Moderado            481450              99.998754
```



9- Relación Compras y Consumo

Se analizó la relación entre el consumo anual de cada producto y la cantidad total comprada durante el año.

Para ello:

- Se calculó el consumo anual por producto.
- Se calculó el total de unidades compradas por producto.
- Se construyó el ratio Compras / Consumo.
- Se midió la correlación entre ambas variables.

Productos analizados: 10.486

Correlación Compras vs Consumo: 0,9987

Clasificación por ratio:

- Alineados (0,9 – 1,1): 4.870 productos
- Sobrecompra (> 1,2): 2.633 productos
- Posible quiebre (< 0,8): 1.803 productos

La correlación cercana a 1 indica una alineación casi perfecta entre compras y consumo.

Esto significa que el sistema de reposición responde adecuadamente a la demanda real de los productos.

Los casos de sobrecompra y posible quiebre se concentran principalmente en productos de bajo consumo, donde pequeñas variaciones generan ratios extremos.

No se observa un desbalance estructural entre lo que se vende y lo que se repone.

```python
# Consumo anual por producto

consumo_prod = (
    balance
    .groupby("Brand")["Consumo"]
    .sum()
    .reset_index()
)

# Compras anuales por producto
compras_prod = (
    Compra_final
    .groupby("Brand")["Quantity"]
    .sum()
    .reset_index(name="Compras")
)

# Unimos
df_cc = compras_prod.merge(consumo_prod, on="Brand", how="inner")

# Limpiamos casos problemáticos (evitar división por 0 y consumos negativos)
df_cc = df_cc[df_cc["Consumo"] > 0].copy()

# Ratio compras/consumo
df_cc["Ratio_Compras_Consumo"] = df_cc["Compras"] / df_cc["Consumo"]

# Correlación Compras vs Consumo (si el consumo sube, ¿compras suben?)
corr = df_cc["Compras"].corr(df_cc["Consumo"])
```

```python
print("Compras vs Consumo (por producto)")
print("Productos analizados:", df_cc["Brand"].nunique())
print("Correlación (Compras vs Consumo):", round(corr, 4))

print("Señales operativas (por ratio)")
print("Alineados (0.9 - 1.1):", ((df_cc["Ratio_Compras_Consumo"]>=0.9)&(df_cc["Ratio_Compras_Consumo"]<=1.1)).sum())
print("Sobrecompra (> 1.2):", (df_cc["Ratio_Compras_Consumo"]>1.2).sum())
print("Posible quiebre (< 0.8):", (df_cc["Ratio_Compras_Consumo"]<0.8).sum())

print("Top 10 sobrecompra (mayor ratio)")
print(df_cc.sort_values("Ratio_Compras_Consumo", ascending=False)
          .head()[["Brand","Compras","Consumo","Ratio_Compras_Consumo"]]
          .to_string(index=False))

print("Top 10 posible quiebre (menor ratio)")
print(df_cc.sort_values("Ratio_Compras_Consumo", ascending=True)
          .head()[["Brand","Compras","Consumo","Ratio_Compras_Consumo"]]
          .to_string(index=False))
```

```
Compras vs Consumo (por producto)
Productos analizados: 10486
Correlación (Compras vs Consumo): 0.9987
Señales operativas (por ratio)
Alineados (0.9 - 1.1): 4870
Sobrecompra (> 1.2): 2633
Posible quiebre (< 0.8): 1803
Top 10 sobrecompra (mayor ratio)
 Brand  Compras  Consumo  Ratio_Compras_Consumo
 27342     1065      3.0                  355.0
 15880      600      2.0                  300.0
 27553      228      1.0                  228.0
 27274      174      1.0                  174.0
 27969      168      1.0                  168.0
Top 10 posible quiebre (menor ratio)
 Brand  Compras  Consumo  Ratio_Compras_Consumo
  4703       10   2745.0               0.003643
 11209        1    171.0               0.005848
  5831        4    684.0               0.005848
  5335        1    159.0               0.006289
  3406        1    158.0               0.006329
```

Conclusión

El análisis integral del proceso de abastecimiento permite concluir que el sistema logístico opera bajo un esquema estructuralmente estable, homogéneo y coherente con la estrategia comercial de la empresa.

En términos de tiempos de entrega, los 126 proveedores analizados presentan una baja dispersión en el Lead Time, con una fuerte concentración entre 7 y 8 días. La clasificación por velocidad evidencia que el 94% de los proveedores se ubica en la categoría Moderado, sin una dependencia significativa de proveedores extremadamente rápidos ni una exposición relevante a proveedores lentos.

Desde la perspectiva de estructura de abastecimiento, se observa un modelo altamente exclusivo: más de 10.600 productos dependen de un único proveedor y solo 29 presentan abastecimiento compartido. Esto implica baja redundancia en la cadena de suministro y una

dependencia individual elevada por producto, aunque sin evidencias actuales de fallas sistémicas.

El cruce entre categoría ABC y velocidad logística confirma que no existe una diferenciación operativa en función de la criticidad del producto. Los productos estratégicos (categoría A) no están asignados a proveedores más rápidos; la mayoría de A, B y C depende de proveedores con tiempos promedio de 8 días. Esto demuestra un modelo logístico homogéneo y no segmentado por importancia estratégica.

Desde el punto de vista económico, el volumen anual de compras se encuentra altamente concentrado en proveedores de categoría Moderado, representando más del 99% del gasto total. Asimismo, el análisis por proveedor evidencia una concentración financiera en un grupo reducido de actores estratégicos, lo cual requiere monitoreo continuo aunque no representa actualmente una vulnerabilidad operativa.

El análisis de frecuencia de compra y cobertura por categoría ABC muestra una clara coherencia entre importancia del producto y política de reposición:

- Los productos A presentan alta frecuencia de compra y cobertura del 100%.
- Los productos B también muestran cobertura total.
- La no reposición se concentra en productos C, de bajo impacto operativo.

Finalmente, el análisis Compras vs Consumo arroja una correlación de 0,9987, lo que indica una alineación prácticamente perfecta entre demanda y reposición. No se detecta un desbalance estructural de sobreabastecimiento ni riesgo sistémico de quiebre de stock. Las desviaciones observadas se concentran en productos de baja rotación y no comprometen la estabilidad general del sistema.

El cruce entre tipo de tienda y velocidad del proveedor confirma que las diferencias de stock entre tiendas no están condicionadas por variaciones significativas en el Lead Time, sino que responden a dinámicas propias de demanda o decisiones operativas internas.

Conclusión General

El sistema de abastecimiento presenta:

- Estabilidad en tiempos de entrega.
- Alta alineación entre compras y consumo.
- Priorización correcta de productos estratégicos.
- Concentración controlada en proveedores moderados.
- Ausencia de riesgos logísticos estructurales.

En términos globales, el proceso de compras y reposición se encuentra optimizado y no evidencia vulnerabilidades críticas en la cadena de suministro.

Bloque 7 – Análisis Financiero del Inventario

Objetivo: El objetivo de esta etapa es traducir el estado del inventario desde una perspectiva operativa hacia una dimensión financiera, cuantificando el impacto del stock sobre el capital de trabajo de la empresa (Working Capital).

Mientras que los bloques anteriores analizaron consumo, rotación y abastecimiento, este bloque busca responder preguntas económicas clave:

- ¿Cuánto capital se encuentra actualmente inmovilizado en inventario?
- ¿Cómo se distribuye ese capital según la importancia estratégica de los productos (ABC)?
- ¿Qué proporción del capital está asociada a productos de baja rotación?
- ¿Existe concentración financiera en determinadas sucursales?

Este análisis permite evaluar no solo la eficiencia operativa del inventario, sino también su impacto directo sobre la liquidez del negocio.

1- Construcción del costo representativo y valorización del stock

Para transformar el inventario en valores monetarios se calculó un costo representativo por producto (Brand), utilizando la mediana histórica del PurchasePrice proveniente del dataset Compra_final.

Posteriormente, se consolidó el stock final por tienda y producto y se calculó el capital inmovilizado mediante:

Capital Inmovilizado = StockFinal × CostRep

Se identificaron:

- 9.289 productos valorizables.
- 4.882.593 unidades en stock final valorizado.
- Capital total inmovilizado: **55.293.243,91 USD**.

Adicionalmente, se detectaron 1.176 productos sin información de costo, los cuales fueron excluidos del cálculo monetario para evitar distorsiones.

Este resultado establece una base financiera sólida para evaluar el inventario en términos de working capital.

```
# Capital inmobilizado

fin_base = stock_final.merge(cost_rep, on="Brand", how="left")

# Control simple: Brands sin costo (no se pueden valorizar)
faltan_costos = fin_base["CostRep"].isna().sum()
if faltan_costos > 0:
    print(f"Filas sin costo (Brand sin compras): {faltan_costos} -> se excluirán del cálculo monetario")

fin_base["CapitalInmovilizado"] = fin_base["StockFinal"] * fin_base["CostRep"]
```

```python
# Datos valorizables
fin_cap = fin_base.dropna(subset=["CostRep"]).copy()

# KPI general
capital_total = fin_cap["CapitalInmovilizado"].sum()
unidades_total = fin_cap["StockFinal"].sum()
skus_valorizables = fin_cap["Brand"].nunique()

print("KPI GENERAL")
print(f"SKUs valorizables: {skus_valorizables:,}")
print(f"Unidades Stock Final: {unidades_total:,.0f}")
print(f"Working Capital (Inventario a costo): ${capital_total:,.2f}")

fin_cap.head()
```

```
Filas sin costo (Brand sin compras): 1176 -> se excluirán del cálculo monetario
KPI GENERAL
SKUs valorizables: 9,289
Unidades Stock Final: 4,882,593
Working Capital (Inventario a costo): $55,293,243.91
```

	Store	Brand	StockFinal	CostRep	CapitalInmovilizado
0	1	58	11	9.28	102.08
1	1	62	7	28.67	200.69
2	1	63	7	30.46	213.22
3	1	72	4	26.11	104.44
4	1	75	7	10.94	76.58

2- Distribución del capital por categoría ABC

Se integró la clasificación ABC previamente calculada para analizar cómo se distribuye el capital según la criticidad económica de los productos.

Resultados obtenidos:

- Categoría A:
  - Capital: 35.296.914,53 USD
  - Participación: 63,83%

- Categoría B:
  - Capital: 11.420.714,38 USD
  - Participación: 20,65%

- Categoría C:
  - Capital: 8.575.615,00 USD
  - Participación: 15,51%

El capital inmovilizado sigue una estructura coherente con la lógica estratégica del negocio. Los productos de categoría A concentran la mayor parte del capital, lo cual es consistente con su mayor impacto económico y volumen operativo.

No se observa una acumulación desproporcionada de capital en productos de baja relevancia (C), lo que sugiere una asignación financiera alineada con la priorización del portafolio.

```python
# Capital por ABC

req_abc = {"Brand", "CategoriaABC"}
if not req_abc.issubset(abc.columns):
    raise KeyError(f"Tu DF 'abc' debe tener {req_abc}. Tiene: {set(abc.columns)}")

# Tabla de mapeo Brand -> CategoriaABC (1 fila por Brand)
abc_map = abc[["Brand", "CategoriaABC"]].drop_duplicates("Brand")

# Merge: asignar categoría ABC a cada fila de fin_cap
fin_abc = fin_cap.merge(abc_map, on="Brand", how="left")

# Agregado: capital por categoría
cap_abc = (
    fin_abc
    .groupby("CategoriaABC", as_index=False)
    .agg(
        Capital=("CapitalInmovilizado", "sum"),
        Unidades=("StockFinal", "sum"),
        SKUs=("Brand", "nunique")
    )
    .sort_values("Capital", ascending=False)
)

cap_abc["ShareCapital"] = cap_abc["Capital"] / cap_abc["Capital"].sum()

print("CAPITAL POR ABC ")
cap_abc
```

```
CAPITAL POR ABC
```

	CategoriaABC	Capital	Unidades	SKUs	ShareCapital
0	A	35296914.53	2967591	1543	0.638359
1	B	11420714.38	1217946	2034	0.206548
2	C	8575615.00	697056	5712	0.155093

3- Identificación de capital asociado a baja rotación

Con el objetivo de evaluar la eficiencia financiera del inventario, se identificaron productos cuya rotación anual se ubica en el cuartil inferior (percentil 25).

Se definió un umbral de baja rotación igual a:

Rotación ≤ 3

Resultados:

- Capital asociado a baja rotación: **17.205.540,44 USD**
- Participación sobre el capital total: **31,12%**

Este indicador revela que aproximadamente un tercio del capital inmovilizado se encuentra en productos con menor dinamismo operativo, lo que implica mayor permanencia en inventario y potencial presión sobre el capital de trabajo.

```python
# Caítal con baja rotacion

req_rot = {"Brand", "Rotacion"}
if not req_rot.issubset(rotacion.columns):
    raise KeyError(f"Tu tabla de rotación debe tener {req_rot}. Tiene: {set(rotacion.columns)}")

fin_rot = fin_cap.merge(rotacion[["Brand", "Rotacion"]], on="Brand", how="left")
fin_rot = fin_rot.dropna(subset=["Rotacion"]).copy()

umbral_rot_p25 = fin_rot["Rotacion"].quantile(0.25)
fin_rot["EsBajaRotacion"] = fin_rot["Rotacion"] <= umbral_rot_p25

capital_lento = fin_rot.loc[fin_rot["EsBajaRotacion"], "CapitalInmovilizado"].sum()
share_lento = capital_lento / capital_total if capital_total else 0

print("CAPITAL LENTO")
print(f"Umbral baja rotación (p25): {umbral_rot_p25:.2f}")
print(f"Capital baja rotación: ${capital_lento:,.2f}")
print(f"Share sobre capital total: {share_lento:.2%}")

fin_rot_abc = fin_rot.merge(abc[["Brand", "CategoriaABC"]], on="Brand", how="left")
capital_lento_C = fin_rot_abc.loc[
    (fin_rot_abc["EsBajaRotacion"]) & (fin_rot_abc["CategoriaABC"] == "C"),
    "CapitalInmovilizado"
].sum()
share_lento_C = capital_lento_C / capital_total if capital_total else 0

print("Capital lento en C:", f"${capital_lento_C:,.2f}")
print("Share lento C / total:", f"{share_lento_C:.2%}")
```

✓ 0.2s

```
CAPITAL LENTO
Umbral baja rotación (p25): 3.00
Capital baja rotación: $17,205,540.44
Share sobre capital total: 31.12%
Capital lento en C: $6,891,428.77
Share lento C / total: 12.46%
```

4. Capital lento dentro de categoría C

Para profundizar el análisis, se evaluó qué proporción del capital lento pertenece a productos de categoría C.

Resultados:

- Capital lento en C: **6.891.428,77 USD**
- Participación sobre el capital total: **12,46%**

Aunque existe capital de baja rotación dentro de categoría C, este no domina la estructura financiera del inventario. Esto refuerza la conclusión previa de que el sistema no presenta un problema estructural de sobrestock, sino oportunidades puntuales de optimización.

5- Concentración del capital por sucursal

Se analizó la distribución del capital inmovilizado a nivel tienda para identificar posibles concentraciones financieras.

Resultados principales:

- Total de tiendas analizadas: 80.
- Cantidad de tiendas que concentran el 80% del capital: **41 sucursales**.

El capital se encuentra distribuido de forma relativamente amplia dentro de la red de sucursales, sin una concentración extrema en pocas ubicaciones. Esto sugiere una asignación de inventario equilibrada desde la perspectiva financiera.

```python
# Concentracion de capital por sucursal

cap_store = (
    fin_cap
    .groupby("Store", as_index=False)
    .agg(
        Capital=("CapitalInmovilizado", "sum"),
        Unidades=("StockFinal", "sum"),
        SKUs=("Brand", "nunique")
    )
    .sort_values("Capital", ascending=False)
)

cap_store["Share"] = cap_store["Capital"] / cap_store["Capital"].sum()
cap_store["ShareAcum"] = cap_store["Share"].cumsum()

tiendas_80 = int((cap_store["ShareAcum"] <= 0.80).sum())

print("TOP 10 SUCURSALES POR CAPITAL")
cap_store.head(10)
```

✓ 0.1s

```
TOP 10 SUCURSALES POR CAPITAL
```

	Store	Capital	Unidades	SKUs	Share	ShareAcum
49	50	3452057.21	260704	7095	0.062432	0.062432
72	73	2225108.69	164402	5389	0.040242	0.102674
66	67	2125269.51	163470	5068	0.038436	0.141110
33	34	2102446.05	144948	6292	0.038024	0.179134
75	76	2046897.61	143689	5069	0.037019	0.216153
68	69	2025102.23	150233	5619	0.036625	0.252777
65	66	1983994.74	144433	4831	0.035881	0.288659
73	74	1966029.29	166015	5199	0.035556	0.324215
37	38	1678120.63	129324	4909	0.030349	0.354565
54	55	1528669.03	125409	4761	0.027647	0.382211

6- Síntesis financiera del inventario

Resumen ejecutivo del bloque:

- Working Capital total en inventario: **55,29 millones USD**.

- 31,12% del capital se encuentra en productos de baja rotación.
- La categoría A concentra la mayor parte del capital, reflejando coherencia estratégica.
- El capital lento no se concentra exclusivamente en productos C, lo que indica que la baja rotación no está limitada a productos marginales.
- La distribución del capital entre tiendas es amplia y sin concentración crítica.

```python
# Resumen ejecutivo

resumen_b7 = pd.DataFrame([{
    "CapitalTotal_WorkingCapital": capital_total,
    "UnidadesStockFinal": unidades_total,
    "SKUs_valorizables": skus_valorizables,
    "CapitalBajaRotacion": capital_lento,
    "ShareCapitalBajaRotacion": share_lento,
    "CapitalBajaRotacion_en_C": capital_lento_C,
    "ShareBajaRotacion_en_C": share_lento_C,
    "Tiendas_80pct_capital": tiendas_80,
    "TotalTiendas": int(cap_store.shape[0])
}])

print("RESUMEN BLOQUE 7")
resumen_b7
```

✓ 0.0s

RESUMEN BLOQUE 7

	CapitalTotal_WorkingCapital	UnidadesStockFinal	SKUs_valorizables	CapitalBajaRotacion	ShareCapitalBajaRotacion	CapitalBajaRotacion_en_C	ShareBajaRotacion_en_C	Tiendas_80pct_capital	TotalTiendas
0	55293243.91	4882593	9289	17205540.44	0.311169	6891428.77	0.124634	41	80

Conclusión del bloque

El análisis financiero evidencia que el inventario presenta una estructura estable y alineada con la lógica ABC del negocio. No se identifican señales de sobreinversión sistémica ni riesgos financieros estructurales.

Sin embargo, la presencia de un 31% del capital asociado a productos de baja rotación señala oportunidades de optimización orientadas a:

- Mejorar la eficiencia del capital de trabajo.
- Reducir tiempos de permanencia en inventario.
- Ajustar políticas de reposición en segmentos de menor dinamismo.

Este bloque complementa el diagnóstico operativo previo, incorporando una perspectiva económica que permite cuantificar el impacto real del inventario sobre la liquidez del sistema.